

# An Improved Hybrid Dual Attack on LWE with Sparse Secrets and its Application to FHE

Lei Bi<sup>1,2</sup>, Yijian Liu<sup>1,2</sup>, Xianhui Lu<sup>1,2</sup>, Junjie Luo<sup>3</sup>, and Kunpeng Wang<sup>1,2</sup>

<sup>1</sup> Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China  
`{bilei, liuyijian, luxianhui, wangkunpeng}@iie.ac.cn`

<sup>2</sup> School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China

<sup>3</sup> School of Mathematics and Statistics, Beijing Jiaotong University, Beijing 100044, China  
`jjluo1@bjtu.edu.cn`

**Abstract.** The Learning with Errors (LWE) problem serves as a cornerstone of modern cryptography, underlying advanced schemes such as Fully Homomorphic Encryption (FHE). Many FHE schemes adopt LWE instances with sparse ternary secrets, leaving them vulnerable to attacks. In 2022, Bi-Lu-Luo-Wang [ACISP 2022] proposed a hybrid dual attack that combines May’s Meet-in-the-Middle (MITM) algorithm [Crypto 2021] with a dual attack and shows that it outperforms other attacks in a large range of FHE-type parameters. However, their attack suffers from two main efficiency bottlenecks: the costly enumeration of error entries and the large number of hash function labels. In this work, we conduct a systematic analysis of several variants of May’s MITM algorithm equipped with different list constructions and hash functions. Based on this, we propose a new hybrid dual attack that incorporates the most efficient variant, effectively mitigating both bottlenecks. We further enhance the attack by adopting a better hypothesis testing algorithm for FHE settings. Addressing recent concerns raised by Ducas-Pulles [Crypto 2023] regarding the independence heuristic in dual attacks, we provide a rigorous theoretical and empirical analysis. We demonstrate that, for typical FHE parameters, our attack does not rely on the problematic independence heuristic and lies outside the contradictory regime. Finally, we compare our attack with previous hybrid attacks, showing consistent and significant improvements across all evaluated cases. In particular, our results invalidate the accelerated BGV scheme in [EUROCRYPT 2024] by reducing its bit-security below the claimed security level, with the most extreme case being 18 bits lower.

**Keywords:** Lattice-based Cryptography · Learning with Errors · Bit-security · Dual Attack · Hybrid Attack.

## 1 Introduction

The Learning with Errors (LWE) problem [39] has emerged as a foundational building block in lattice-based cryptography. The LWE problem asks to recovery

a secret vector  $\mathbf{s}$  given samples  $(\mathbf{A}, \mathbf{b})$ , where  $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$  and  $\mathbf{e}$  is a small error vector. A variety of cryptographic schemes, from homomorphic encryptions (HE), public key encryptions to digital signatures, base their security on the family of the lattice problems. The LWE is widely believed to be quantum-safe and benefits from strong worst-case to average-case reductions, which means that it is at least as hard as some worst-case lattice problems. However, in practice, the bit-security of concrete LWE-based schemes depends crucially on parameter selection, making the precise evaluation of their security levels an essential task.

For efficiency reasons, many practical LWE-based schemes employ small and sparse secrets. As one of the most important implementations of LWE, the fully HE (FHE) schemes use sparse ternary secrets because the bootstrapping procedure, the key technique in FHE, relies on such sparsity for computational efficiency. However, the use of sparsity and small secrets makes this variant of the LWE problem more vulnerable to attacks, and the bit-security of LWE with small/sparse secrets remains insufficiently understood [1]. Recently, the sparse-secret encapsulation technique [7] was proposed to improve the precision and efficiency of bootstrapping without compromising the security of the schemes. Since then lots of new proposed schemes have adopted sparse ternary secrets. Moreover, most widely used FHE libraries, including HEAAN, HELib, Lattigo, and PALISADE [18, 19, 27, 36], adopt sparse secrets, and the Homomorphic Encryption Standardization [22] has listed LWE with sparse secrets as an extension of the current document. These developments make it both important and urgent to rigorously assess the bit-security of LWE with small/sparse secrets.

Estimating the concrete bit-security of LWE-based schemes requires assessing their resilience against all known attacks. For FHE-type parameters, the two most powerful attack families are hybrid primal attacks [42, 20] and hybrid dual attacks [10, 5]. In this paper, we focus on the hybrid dual attack, which currently represents one of the most efficient approaches for FHE-type parameters. A hybrid dual attack partitions the secret vector into two parts:  $\mathbf{s} = (\mathbf{s}_1, \mathbf{s}_2) \Rightarrow \mathbf{A}_2\mathbf{s}_2 + \mathbf{e} = \mathbf{b} - \mathbf{A}_1\mathbf{s}_1 \bmod q$ , then enumerates candidates for  $\mathbf{s}_1$  while using short vectors in the dual lattice of  $\mathbf{A}_2$  to perform hypothesis testing on the guesses. Currently, the most efficient hybrid dual attack on FHE is due to Bi-Lu-Luo-Wang [5], which incorporates May’s Meet-LWE algorithm [31], an optimal Meet-in-the-Middle (MITM) method for solving sparse ternary LWE, to accelerate the enumeration of  $\mathbf{s}_1$ . However, embedding Meet-LWE into the hybrid dual framework amplifies its two inherent weaknesses with respect to error-guessing and hash labels. Consequently, although the Bi-Lu-Luo-Wang algorithm improves bit-security estimates in across a wide range of FHE parameters, it still faces two significant efficiency drawbacks: (1) the high computational cost of error enumeration, and (2) the large number of hash function labels required. Addressing these two issues is critical for further advancing the performance of hybrid dual attacks leveraging Meet-LWE.

In addition, a broader question concerning the effectiveness and reliability of (hybrid) dual attacks was raised by Ducas-Pulles [13] in 2023. They argued that the analysis of these attacks relies on unverified heuristics, particularly

the independence heuristic, and lacks sufficient theoretical and experimental justification. More importantly, they showed that some conclusions derived from these heuristics contradict unconditional lattice-theoretic results. This motivates a careful theoretical and empirical study to determine whether the independence assumptions hold for typical FHE parameter regimes under hybrid dual attacks.

### 1.1 Our Contribution

In this paper, we revisit the Meet-LWE attack and propose a more effective hybrid dual attack for LWE with sparse ternary secrets. Notably, we prove that our attack does not rely on the debated independence heuristic. Our contributions are three-fold.

**(1) Improved Understanding of Meet-LWE and Its Variants.** We thoroughly revisit Meet-LWE and two of its recent enhancements: the new list construction and hash function techniques (Sections 3 and 4). In particular, we simplify the analysis of the list construction method introduced in [20] by adapting it to the pure Meet-LWE setting, and we develop a more theoretically rigorous method for parameter selection that removes the previous reliance on empirical heuristics. We then perform an in-depth comparison of four Meet-LWE variants obtained by combining different list constructions and hash functions. Our analysis reveals several key insights. Surprisingly, while the new list construction from [20] successfully eliminates error enumeration, it frequently results in worse performance across a wide parameter range due to a fundamental structural bottleneck. In contrast, the new randomized hash function from [31, 20] consistently outperforms Odlyzko’s classic hash function [21, 34] by reducing the large number of hash labels at a minor repetition cost. This advantage becomes particularly pronounced with large error distributions, which are typical in hybrid dual attacks. When used together, these two techniques complement each other effectively: the hash function mitigates the main bottleneck of the list construction, while the list construction amplifies the hash function’s impact. As a result, their combination achieves the best overall performance in most practical parameter regimes, though some regions still leave room for further optimization. Contrary to the claims of [24], our findings indicate that it is the new list construction, rather than the new hash function, that primarily resolves the error enumeration issue. The hash function instead addresses the amplified label explosion problem. We believe that our detailed analysis of these techniques contributes to a deeper understanding of how these techniques work in the complex scenarios of MITM attack and hybrid dual attack, and lays the groundwork for further improvements.

**(2) A More Effective Hybrid Dual Attack.** In Section 5, we propose an improved hybrid dual attack based on the Bi-Lu-Luo-Wang (BLLW) algorithm [5], equipped with the above two new techniques. Based on our previous analysis, this change helps to overcome the two main drawbacks of [5]. In addition, regarding the hypothesis testing, we replace the trivial decoder used in previous MITM-accelerated hybrid dual attacks [10, 5] by a better hypothesis testing algorithm referred to as Simple-decoder in [25]. Focused on LWE with

FHE-type parameters, the Gaussian distribution in hybrid dual attack to be distinguished has a relevant small deviation (compared with those in PQC-type parameters) and can be easily distinguished from a uniform distribution. Therefore, both [10] and [5] simply selects a threshold to decide the distribution. However, this trivial decoder requires a large dimension to ensure a high success probability, which increases the complexity of the former MITM process. This inefficiency has been overlooked by previous MITM-accelerated hybrid dual attacks [10, 5]. In our new attack we replace the trivial decoder with the Simple-decoder, which can decrease the dimension and thus decrease the complexity while maintaining a similar success probability.

**(3) Clarifying the Independence Heuristic.** In response to recent concerns raised by Ducas and Pulles [13], we clarify that our attack is not affected by it through both theoretical analysis and experiments (Section 6). The core reason for this is the small ratios of  $\frac{\sigma}{q}$  and  $\frac{w}{n}$  in FHE schemes (where  $\sigma$  is the standard deviation of the error and  $w$  is the hamming weight of the secret), which simplify the instances requiring hypothesis testing and decrease the number of testing instances. As it is easy to distinguish the correct one, we only need a small number of short vectors from the dual lattice for the hypothesis testing. Indeed, the number is smaller than the dimension of the lattice, so we can use linearly independent ones to do the testing, which makes the scores independent from each other. We analyze and experimentally verify this argument to show that our attack is out of the scope of the contradictory regime of independence heuristic.

Finally, in Section 7 we compare our new attack with previous MITM-accelerating hybrid dual attacks [5, 10] on a series of LWE instances with FHE-type parameters. The results show that our new attack offers significant improvements across all cases. Particularly, we estimate the bit-security of CKKS [11], a leading scheme used in many open-source implementations of HE. Our attack reduces the bit-security by 17-24 bits compared with [5, 10], bringing the estimated security level close to the claimed boundary (82 bits vs 80 bits). Additionally, we estimate the security of the accelerated BGV schemes published in EUROCRYPT24 [29], showing that the bit-security falls below the claimed security level by up to 18 bits, thus invalidating the schemes and requiring re-parameterized. We remark that our results do not invalidate the security of LWE-based schemes in NIST-PQC as MITM-accelerating hybrid dual attacks do not fit their parameters.

## 1.2 Related Work

**MITM Attack.** Odlyzko’s [21] and Howgrave-Graham’s [34] MITM attacks are two basic forms of meet-in-the-middle with different splitting methods of the secret:  $\mathbf{As} = \mathbf{A}_1\mathbf{s}_1 + \mathbf{A}_2\mathbf{s}_2$  vs  $\mathbf{As} = \mathbf{A}(\mathbf{s}_1 + \mathbf{s}_2)$ . Odlyzko’s algorithm runs in time  $O(S^{0.5})$ , where  $S$  is the search space of  $\mathbf{s}$  and Howgrave-Graham’s is worse than that. In 2021, May [31] proposed a new MITM attack named Meet-LWE on sparse ternary LWE, reduces the complexity to  $O(S^{0.25})$ . This work is followed up by a series of improvements [15, 16, 20, 23, 24, 28] and among them,

Kirshanova-May [24] and Hhan-Kim-Lee-Son [20] independently tried to remove the two auxiliary overheads by using similar techniques and Hhan-Kim-Lee-Son [20] combined the improved Meet-LWE into the hybrid primal attack.

**Hybrid Dual Attack.** The first hybrid dual attack was introduced by Albrecht [2] in 2017. After that, this type of attacks evolves rapidly and has become the most efficient bit-security estimation algorithm of LWE. In 2020, Espitau-Joux-Kharchenko [14] introduced a FFT-type distinguisher and an efficient matrix multiplication method to accelerate guessing small secrets. Bi-Lu-Luo-Wang-Zhang [6] further generalized the matrix multiplication method for LWE with arbitrary secrets, and showed that hybrid dual attack always outperforms the standalone dual attack for most cryptographic use cases regardless of the secret distribution. In 2021, Guo-Johansson [17] gave a hybrid dual attack by introducing a FFT distinguisher and a two-step lattice-reduction which is followed quickly by an independent report of MATZOV [30], which further improves the attack by combining it with the modulus switching technique. Two follow up works by Albrecht-Shen [3] and Carrier-Shen-Tillich [8] improve this attack by using quantum algorithm and coding theory, respectively.

For LWE with FHE-type parameters, another branch of hybrid dual attacks using MITM techniques further improves the efficiency. Cheon-Hhan-Hong-Son [10] introduced a hybrid dual attack for LWE with FHE-type parameters by using Howgrave-Graham’s MITM attack [34], which significantly improves the performance of the original hybrid dual attack by Albrecht [2]. After that, Bi-Lu-Luo-Wang [5] continue to improve this type of attack by replacing Howgrave-Graham’s MITM attack with Meet-LWE, a more efficient MITM attack proposed by May [31].

**Hybrid Primal Attack.** The hybrid primal attack on LWE was first presented by Wunderer [42], which combines the primal attack with Howgrave-Graham’s MITM attack [34]. This attack works for LWE with arbitrary secret and error. Afterwards, Son-Cheon [40] showed that when focusing on LWE with FHE-type parameters, Wunderer’s hybrid primal attack can be further improved because of the structure of the sparse ternary secret. Recently, Hhan-Kim-Lee-Son [20] proposed a new hybrid primal attack on FHE that combines primal attack and a generalization of May’s Meet-LWE attack [31] using a new list construction.

## 2 Preliminaries

### 2.1 Notations

Denote  $\mathcal{U}(S)$  the uniform distribution over a compact set  $S \in \mathbb{R}^n$ . Denote  $\mathcal{G}_{\mathbf{c},\sigma}$  a Gaussian distribution with center  $\mathbf{c}$  and standard deviation  $\sigma$ , and use  $\mathcal{G}_\sigma$  as shorthand for  $\mathcal{G}_{\mathbf{0},\sigma}$ . The multinomial coefficient is denoted as  $\binom{a}{b_1, b_2} := \binom{a}{b_1} \binom{a-b_1}{b_2}$ . The Gauss error function is defined as  $\text{erf}(x) := \frac{2}{\sqrt{\pi}} \int_{-\infty}^x e^{-t^2} dt$ . We write  $[n] := \{1, \dots, n\}$ .

## 2.2 Lattice and Lattice Reduction

A lattice is a discrete additive subgroup of  $\mathbb{R}^m$  for some  $m \in \mathbb{N}$ . Formally, a lattice  $\Lambda$  is the set of all integer linear combinations of a set of  $n$  linearly independent vectors  $\mathbf{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_n\} \subset \mathbb{R}^m$ , i.e.,  $\Lambda = \{\sum_{i \in [n]} z_i \cdot \mathbf{b}_i : z_i \in \mathbb{Z}\}$ . The determinant of the lattice is given by  $\det(\Lambda) = \sqrt{\det(\mathbf{B}^T \mathbf{B})}$ . For a random lattice  $\Lambda$  of dimension  $m$ , the Gaussian heuristic estimates the norm of the shortest vector in the lattice  $\lambda_1(\Lambda)$  as  $\text{GH}(\Lambda) \approx \det(\Lambda)^{1/m} \cdot \sqrt{\frac{m}{2\pi e}}$ .

A lattice reduction algorithm takes a basis of a lattice as input and returns a new basis of the lattice with higher quality. The quality of a basis is characterized by the root-Hermit-factor  $\delta_0$ , defined as  $\delta_0^m = \|\mathbf{b}_1\| / \det(\Lambda)^{\frac{1}{m}}$ , where  $\mathbf{b}_1$  is the first (and shortest) vector in the output basis. BKZ algorithm is exploited for lattice reduction. It takes a blocksize  $\beta$  and a basis as input, and outputs a reduced basis with root-Hermit-factor  $\delta_0 = (\frac{\beta}{2\pi e} (\pi\beta)^{\frac{1}{\beta}})^{\frac{1}{2(\beta-1)}}$ .

(Hybrid) Dual attacks utilize the BKZ algorithm to generate short vectors. In this paper, following the approach in [17, 30], we adopt a two-step reduction strategy: (1) apply BKZ with block size  $\beta_1$  to the dual lattice; (2) perform sieving on the lattice  $L$  spanned by the first  $\beta_2$  vectors of the reduced basis, yielding  $N_{\text{sieve}}(\beta_2)$  short vectors with norm at most  $\sqrt{4/3} \cdot \lambda_1(L)$ . Let  $M$  denote the number of short vectors used in the hybrid dual attack. These are obtained by selecting the  $M$  shortest vectors among the  $N_{\text{sieve}}(\beta_2)$  generated ones.

In addition, we follow [9, 14] to assume that the coordinates of the vectors output by BKZ are approximately independently distributed according to a Gaussian distributions with mean 0 and standard deviation  $\ell/\sqrt{d}$ , where  $d$  is the lattice dimension.

## 2.3 The Learning with Errors Problem

**Definition 1 (LWE [39]).** Let  $n, q \in \mathbb{N}$ .  $\mathcal{S}$  is the secret distribution over  $\mathbb{Z}_q^n$  and  $\chi$  is a small error distribution over  $\mathbb{Z}$ . For a secret  $\mathbf{s} \leftarrow \mathcal{S}$ , denote  $\text{LWE}_{n,q,\mathbf{s},\chi}$  the probability distribution on  $\mathbb{Z}_q^n \times \mathbb{Z}_q$  obtained by sampling  $\mathbf{a} \in \mathbb{Z}_q^n$  uniformly random, sampling  $e \leftarrow \chi$  and returning  $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ . Given access to the outputs from  $\text{LWE}_{n,q,\mathbf{s},\chi}$ , we define two versions of LWE problem. *Decision-LWE*: Given  $m$  instances, distinguish  $\text{LWE}_{n,q,\mathbf{s},\chi}$  from  $\mathcal{U}(\mathbb{Z}_q^n \times \mathbb{Z}_q)$  for a fixed  $\mathbf{s} \leftarrow \mathcal{S}$ . *Search-LWE*: Given  $m$  instances sampled from  $\text{LWE}_{n,q,\mathbf{s},\chi}$  with a fixed  $\mathbf{s} \leftarrow \mathcal{S}$ , recover  $\mathbf{s}$ .

We rewrite the LWE instances into matrix form as  $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q)$  where  $\mathbf{s} \leftarrow \mathcal{S}$ ,  $\mathbf{A} \leftarrow \mathbb{Z}_q^{m \times n}$ ,  $\mathbf{e} \leftarrow \chi^m$ ,  $\mathbf{b} \in \mathbb{Z}_q^m$ . In this work, we focus on sparse ternary LWE and define two distributions for it as follows.

**Ternary-0:** Given an even number  $w$ , let  $\mathcal{T}_0^n(w)$  denote the set of all vectors  $\mathbf{s} \in \{0, \pm 1\}^n$  that have exactly  $\frac{w}{2}$  entries of 1 and  $\frac{w}{2}$  entries of  $-1$ . The uniform distribution over  $\mathcal{T}_0^n(w)$ , denoted by  $\mathcal{U}_0^n(w)$ , assigns equal probability to each vector in this set.

**Ternary-1:** Given an even number  $w$ , let  $\mathcal{T}_1^n(w) = \bigcup_{h=0, \text{even}}^w \mathcal{T}_0^n(h)$ , i.e.,  $\mathcal{T}_1^n(h)$  is the set of all vectors  $\mathbf{s} \in \{0, \pm 1\}^n$  that have exactly  $\frac{h}{2}$  entries of 1

and  $\frac{h}{2}$  entries of  $-1$ , where  $h$  is an even number with  $0 \leq h \leq w$ . Let  $p$  be a probability mass function defined on the even integers  $h$  such that  $0 \leq h \leq w$ , with  $\sum_{h=0}^w p(h) = 1$ . Given  $p$  and  $w$ , the mixture distribution over  $\mathcal{T}_1^n(w)$ , denoted by  $\mathcal{U}_1^n(w, p)$ , is defined as follows: A vector is drawn by first selecting  $h$  with probability  $p(h)$ , and then uniformly sampling a vector from  $\mathcal{T}_0^n(h)$ .

## 2.4 Lemmas

**Lemma 1 ([40, 20]).** *Let  $x \leftarrow \mathcal{U}[0, B)$  and  $e \leftarrow \chi$  with  $\text{supp}(\chi) = [-E, E]$ . If  $\chi$  is a Gaussian distribution, then the probability that  $x + e \in [0, B)$  is*

$$p_c(E, B) := \Pr[x + e \in [0, B) | x \leftarrow \mathcal{U}[0, B), e \leftarrow \mathcal{G}_E] = \text{erf}(\tau) + \frac{e^{-\tau^2} - 1}{\sqrt{\pi}\tau},$$

where  $\tau = \frac{B}{\sqrt{2E}}$ . If  $\chi$  is a uniform distribution, then

$$p_c(E, B) := \Pr \left[ x + e \in [0, B) \left| \begin{array}{l} x \leftarrow \mathcal{U}[0, B) \\ e \leftarrow \mathcal{U}[-E, E) \end{array} \right. \right] = \begin{cases} \frac{1}{2E} & \text{for } B = 0 \\ \frac{B}{2E} & \text{for } 0 < B < \frac{E}{2} \\ 1 - \frac{E}{2B} & \text{for } B \geq \frac{E}{2} \end{cases}.$$

**Lemma 2 ([13]).** *Let  $\Lambda$  be a lattice with volume 1 and  $r > 0$  such that  $r < \frac{\lambda_1(\Lambda)}{2GH(\Lambda)}$ . Then, for a target  $\mathbf{t}$  uniform in  $\mathbb{R}^m/\Lambda$ , it holds with probability  $r^m$  that  $\mathbf{t}$  is at distance at most  $r \cdot GH(\Lambda)$  from the lattice.*

## 3 Meet-LWE attack and Two New Techniques

In this section, we begin by reviewing the original Meet-LWE attack proposed by [31] and analyze its limitations when applied in a hybrid dual attack (Section 3.1). Then we revisit two recent techniques that enhance Meet-LWE from two perspectives: hash function and list construction (Section 3.2 and 3.3, respectively). In particular, for the list construction technique originally introduced in the context of a hybrid primal attack [20], we present a simplified analysis tailored to the pure Meet-LWE setting. Our contribution includes a more theoretically grounded method for parameter selection, which avoids reliance on empirical heuristics and improves the general applicability of the approach.

### 3.1 Meet-LWE Attack

Consider an LWE instance  $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^n$ , where  $\mathbf{s} \in \mathcal{T}_0^n(w)$  and  $\mathbf{e} \in \chi^m$ , with  $\text{supp}(\chi) = [-E, E]$ .<sup>4</sup> A typical MITM attack solves

<sup>4</sup> When  $\chi = \mathcal{U}$ ,  $\mathbf{e}$  is uniform in  $[-E, E]^m$ . When  $\chi = \mathcal{G}_\sigma$ , we treat its effective range as  $[-3\sigma, 3\sigma]^m$ . This ensures that  $\mathbf{e}$  falls within this range with overwhelming probability. Note that the success of Meet-LWE depends only on the range of the error, not on its precise distribution.

the LWE instance by splitting the equation as  $\mathbf{As}_1 = \mathbf{b} - \mathbf{As}_2 - \mathbf{e} \bmod q$ , where  $\mathbf{s}_1 + \mathbf{s}_2 = \mathbf{s}$  with  $\mathbf{s}_1, \mathbf{s}_2 \in \mathcal{T}_0^n(\frac{w}{2})$ , followed by enumerating  $(\mathbf{s}_1, \mathbf{s}_2)$  to find a pair  $(\mathbf{As}_1, \mathbf{b} - \mathbf{As}_2)$  that differs from each other by  $\mathbf{e}$ . The main idea of Meet-LWE is that  $\mathbf{s}$  can be represented as the sum of  $\mathbf{s}_1$  and  $\mathbf{s}_2$  in  $R = \binom{w/2}{w/4}^2$  different ways, thus in expectation we can find one of them by only enumerating  $\frac{1}{R}$ -fraction of the overall search space of  $(\mathbf{s}_1, \mathbf{s}_2)$ .

Meet-LWE filters out the  $\frac{1}{R}$ -fraction of the search space in the following way. Let  $k = \lfloor \log_q R \rfloor$  and define a projection mapping  $\pi_k^m(\cdot)$  to extract the first  $k$  coordinates of a  $m$ -length vector. We choose a random target  $\mathbf{t} \in \mathbb{Z}_q^k$  and only enumerate  $(\mathbf{s}_1, \mathbf{s}_2)$  that satisfy the following condition:

$$\pi_k^m(\mathbf{As}_1 + \mathbf{e}) = \pi_k^m(\mathbf{b} - \mathbf{As}_2) = \mathbf{t} \bmod q. \quad (1)$$

As  $\pi_k^m(\mathbf{As}_1 + \mathbf{e})$  and  $\pi_k^m(\mathbf{b} - \mathbf{As}_2)$  can be regarded as uniformly distributed over  $\mathbb{Z}_q^k$ , those that satisfy Eq.(1) account for  $\frac{1}{q^k} \approx \frac{1}{R}$  of the entire set. In this way, we get one solution  $(\mathbf{As}_1, \mathbf{b} - \mathbf{As}_2)$  with high probability by enumerating  $\frac{1}{R}$  of the search space of  $(\mathbf{s}_1, \mathbf{s}_2)$ .

There is an additional expense in this process: enumerating the error  $\pi_k^m(\mathbf{e})$  in the first term  $\pi_k^m(\mathbf{As}_1 + \mathbf{e})$ . Meet-LWE splits the error as  $\pi_k^m(\mathbf{e}) = \pi_k^m((\mathbf{e}_1|\mathbf{0})) + \pi_k^m((\mathbf{0}|\mathbf{e}_2))$  through standard MITM manner to decrease the time complexity. Overall, we construct the following two lists:<sup>5</sup>

$$\begin{aligned} L_1 &= \{\mathbf{s}_1 : \pi_k^m(\mathbf{As}_1 + (\mathbf{e}_1|\mathbf{0})) = \mathbf{t} \bmod q\}, \\ L_2 &= \{\mathbf{s}_2 : \pi_k^m(\mathbf{b} - \mathbf{As}_2 + (\mathbf{0}|\mathbf{e}_2)) = \mathbf{t} \bmod q\}. \end{aligned}$$

The pairs in  $L_1 \times L_2$  satisfy the LWE equation in the first  $k$  entries. It remains to find a pair  $(\mathbf{As}_1, \mathbf{b} - \mathbf{As}_2)$  such that they are close to each other in the remaining  $m - k$  entries. In the original Meet-LWE, this is done by applying Odlyzko's hash function, which is defined as

$$h(\mathbf{x})_i = \begin{cases} 0 & \text{if } x_i \in [-\frac{q}{2}, -E) \\ 1 & \text{if } x_i \in [0, \frac{q}{2} - E) \\ 0, 1 & \text{if } x_i \in [-E, 0) \cup [\frac{q}{2} - E, \frac{q}{2}) \end{cases}. \quad (2)$$

Note that border values in  $[-E, 0) \cup [\frac{q}{2} - E, \frac{q}{2})$  will get both labels. Let  $|L|$  denote the size of lists and  $d$  denote the dimension of coordinates to perform the hash function. Then the total number of hash labels for one list is  $\overline{|L|} := |L| \cdot 2^{\frac{2E+1}{q}d}$  and the expected number of collision between two such lists is  $2^{-d}(\overline{|L|})^2$ . Hence, the overall time complexity is

$$T_{\text{Odlyzko}}(|L|, d) = \max\{\overline{|L|}, 2^{-d}(\overline{|L|})^2\}. \quad (3)$$

While Meet-LWE effectively reduces the search space by exploiting multiple representations of the secret, it also introduces two time-consuming operations:

<sup>5</sup> These lists can be constructed in a standard MITM manner, e.g., enumerate  $\mathbf{s}_1$  as the sum of  $\mathbf{u}_1 \in \mathcal{T}_0^{n/2}(\frac{w}{4}) \times 0^{n/2}$  and  $\mathbf{u}_2 \in 0^{n/2} \times \mathcal{T}_0^{n/2}(\frac{w}{4})$ .

**Algorithm 1** Use LSH-hash to find near pairs

---

**Require:** lists  $L_1, L_2$  of size  $|L|$ ,  $E$   
**Ensure:** A set of  $E$ -near pairs  $(\mathbf{x}, \mathbf{x}') \in L_1 \times L_2$   
1: choose a range  $B$   
2: **repeat**  
3:   choose  $\mathbf{u} \leftarrow \mathbb{Z}_q^m$   
4:   apply  $h_{B, \mathbf{u}}$  to  $L_1, L_2$  and sort  $L_1, L_2$  according to the hash values  
5:   merge the sorted lists according to their hash values  
6:   output the pairs  $(\mathbf{x}, \mathbf{x}') \in L_1 \times L_2$  satisfying  $\|\mathbf{x} - \mathbf{x}'\|_\infty \leq E$   
7: **until**  $\varphi$  times

---

(1) enumerating the  $k$ -entries of the error vector, and  
(2) computing a large number of hash labels for values near the boundary.  
These operations become especially costly when the error range  $E$  is large, as is typical in the hybrid dual attack scenario. How to avoid these inefficiencies was raised as an open problem in [31, 5]. Next we revisit two recently introduced proposed techniques designed to address these challenges.

### 3.2 Randomized Hash Function

We begin by reviewing the improved hash function that can avoid the issue of excessive hash labels near boarder values in Odlyzko's hash function. The idea is to partition the  $\mathbb{Z}_q$  torus into multiple segments and, more importantly, to introduce randomization into the partitioning process [33]. We refer to Odlyzko's original hash function, used in the original Meet-LWE, as Hash1 (see Eq.(2)), and the improved version as Hash2. We remark that Hash2 was first proposed and analyzed in the Meet-LWE variant of Kirshanova-May [24], and was later also used in the work of Hhan-Kim-Lee-Son [20].

Given a selected bucket size  $B$  and a uniformly random shift vector  $\mathbf{u} \in \mathbb{Z}_q^d$ , define a locality-sensitive hash function (LSH) as

$$h_{\mathbf{u}, B} : \mathbb{Z}_q^d \rightarrow [0, \dots, \theta - 1]^d,$$

$$(x_1, \dots, x_d) \mapsto (\lfloor \frac{x_1 + u_1}{B} \rfloor \bmod \theta, \dots, \lfloor \frac{x_d + u_d}{B} \rfloor \bmod \theta),$$

where  $\theta = \lceil \frac{q}{B} \rceil$ . Let  $H_B^d = \{h_{\mathbf{u}, B} | \mathbf{u} \in \mathbb{Z}_q^d\}$  denote a family of the above LSH-hash functions parameterized by  $B$  and  $\mathbf{u}$ . The algorithm of using this LSH-hash to find near pairs is shown in Algorithm 1. Here we name a pair of vectors  $(\mathbf{x}, \mathbf{x}')$  as an  $E$ -near pair if  $\|\mathbf{x} - \mathbf{x}'\|_\infty \leq E$ .

For an  $E$ -near pair  $(\mathbf{x}, \mathbf{x}')$ , the probability that they have the same hash value is  $p_{\text{near}} := \Pr[h_{B, \mathbf{u}}(\mathbf{x}) = h_{B, \mathbf{u}}(\mathbf{x}')] = p_c(E, B)^d$ , as established in Lemma 1. By repeating the procedure (Line 3-6 in Algorithm 1) for  $\varphi$  times, the algorithm finds the pair with probability at least  $p_{\text{LSH}} = 1 - (1 - p_{\text{near}})^\varphi$ . To ensure that the algorithm finds a  $(1 - o(1))$ -fraction of all  $E$ -near pairs with high probability, we follow [20] and set  $\varphi = \frac{10}{p_{\text{near}}}$ . The total number of hash evaluations is  $\varphi \cdot |L|$ ,

and the expected number of collisions is approximately  $\varphi \cdot \frac{|L|^2}{2} \cdot \frac{1}{\theta^d}$  [20]. Then the overall runtime of the algorithm is computed as

$$T_{\text{LSH}}(|L|, d, B) \approx \max\{|L|, \frac{|L|^2}{2} \cdot \frac{1}{\theta^d}\} \cdot \frac{10}{p_c(E, B)^d}, \quad (4)$$

where  $\theta = \lceil \frac{q}{B} \rceil$ , and the optimal  $B$  is found by enumeration. From this runtime expression, we observe that Hash2 avoids the issue of excessive hash labels at the cost of introducing a repetition number  $\varphi$ .

### 3.3 List Construction

Next we revisit a recent list construction technique designed to eliminate the error enumeration. The challenge of removing error enumeration was first posed as an open problem by May [31]. In response, Kirshanova-May [24] introduced a solution that combined a new list construction with a randomized hash function. However, as later noted in [20, Appendix A], their success probability analysis contains a flaw. When corrected, the resulting algorithm becomes impractical.

To address this, Hhan-Kim-Lee-Son [20] proposed a new list construction that also avoids error enumeration and incorporated it into the hybrid primal attack framework. However, due to the complexity introduced by the hybrid setting, their analysis is somewhat intricate and relies on empirical methods for parameter selection.

In this subsection, we revisit the list construction of Hhan-Kim-Lee-Son [20] and present an alternative, simplified analysis within the context of a pure Meet-LWE attack. In particular, we show how to choose parameters analytically using a simple worst-case probability criterion, thereby eliminating the reliance on experimental heuristics. For simplicity, we focus on the case with four levels of lists, which suffices to illustrate the core ideas of the construction and its analysis. For clarity of reference, we denote the original list construction used in Meet-LWE as Const.1 and the variant introduced in [20] as Const.2.

**Lists.** For the level-1 lists, instead of choosing a fixed target  $\mathbf{t} \in \mathbb{Z}_q^k$  as in Const.1, Const.2 chooses an interval  $[-\ell^{(1)}, \ell^{(1)}]$  centered around the origin on  $k^{(1)}$  coordinates to ensure that approximately a  $\frac{1}{R^{(1)}}$ -fraction of the total search space for  $(\mathbf{s}_1^{(1)}, \mathbf{s}_2^{(1)})$  is captured in the lists:

$$\pi_k^m(\mathbf{A}\mathbf{s}_1^{(1)}) \in [-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}} \text{ and } \pi_k^m(\mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)}) \in [-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}}, \quad (5)$$

where  $\mathbf{s}_1^{(1)}, \mathbf{s}_2^{(1)} \in \mathcal{T}_0^n(\frac{w}{2})$ . Note that setting  $\ell^{(1)} = 0$  reduces this condition to the one used in Const.1 with  $\mathbf{t} = \mathbf{0}$ , making Const.1 a special case of Const.2. The level-1 lists are then defined as:

$$\begin{aligned} L_1^{(1)} &= \{(\mathbf{s}_1^{(1)}, \mathbf{A}\mathbf{s}_1^{(1)}) : \pi_{k^{(1)}}^m(\mathbf{A}\mathbf{s}_1^{(1)}) \in [-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}}\}, \\ L_2^{(1)} &= \{(\mathbf{s}_2^{(1)}, \mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)}) : \pi_{k^{(1)}}^m(\mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)}) \in [-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}}\}. \end{aligned}$$

Recall that in Const.1, to ensure Eq.(1), we must enumerate over  $\pi_{k^{(1)}}^m(\mathbf{e}) \in [-E, E]^{k^{(1)}}$ , which incurs significant computational cost. Const.2 avoids this enumeration by instead introducing a probability that combining  $\mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)}$  and  $\mathbf{A}\mathbf{s}_1^{(1)}$  satisfying Eq.(5) can get the desired error  $\mathbf{e}$ . We will analyze how this influence the selection of  $(k^{(1)}, \ell^{(1)})$  later.

These level-1 lists are constructed from level-2 lists, which are defined in a similar manner. Given proper parameters  $(k^{(2)}, \ell^{(2)})$ , level-2 lists are defined as:

$$\begin{aligned} L_i^{(2)} &= \{(\mathbf{s}_i^{(2)}, \mathbf{A}\mathbf{s}_i^{(2)}) : \pi_{k^{(2)}}^m(\mathbf{A}\mathbf{s}_i^{(2)}) \in [-\ell^{(2)}, \ell^{(2)}]^{k^{(2)}}\}, i = 1, 2, 3, \\ L_4^{(2)} &= \{(\mathbf{s}_4^{(2)}, \mathbf{b} - \mathbf{A}\mathbf{s}_4^{(2)}) : \pi_{k^{(2)}}^m(\mathbf{b} - \mathbf{A}\mathbf{s}_4^{(2)}) \in [-\ell^{(2)}, \ell^{(2)}]^{k^{(2)}}\}, \end{aligned}$$

where  $\mathbf{s}_i^{(2)} \in \mathcal{T}_0^n(\frac{w}{4})$  ( $i = 1, 2, 3, 4$ ). Note that these lists must ensure that for each element in  $L_i^{(1)}$  ( $i = 1, 2$ ), at least one out of  $R^{(2)}$  representations of  $\mathbf{s}_i^{(1)} = \mathbf{s}_{2i-1}^{(2)} + \mathbf{s}_{2i}^{(2)}$  exists in the level-2 lists. This requirement guides the selection of  $(k^{(2)}, \ell^{(2)})$ , which we will analyze in detail later.

Finally, level-2 lists are constructed from the level-3 (top level) lists  $L_i^{(3)}$ , which are defined using Odlyzko's MITM technique.<sup>6</sup> In this construction, each vector  $\mathbf{s}_i^{(2)} \in \mathcal{T}_0^n(\frac{w}{4})$  is enumerated as the sum of  $\mathbf{s}_{2i-1}^{(3)} \in \mathcal{T}_0^{n/2}(\frac{w}{8}) \times 0^{\frac{n}{2}}$  and  $\mathbf{s}_{2i}^{(3)} \in 0^{\frac{n}{2}} \times \mathcal{T}_0^{n/2}(\frac{w}{8})$ .

**Algorithm.** The complete Meet-LWE algorithm using Const.2 is summarized in Algorithm 2. The selection of parameters  $(k^{(i)}, \ell^{(i)})$  in step 1 will be analyzed in detail later. The algorithm begins by generating the top-level lists and then recursively constructs lower-level lists by applying hash function to selected coordinates. In step 4, the hash function is applied to the  $k^{(1)} - k^{(2)}$  coordinates for two main reasons. First, we aim to find  $\ell^{(1)}$ -near collisions in  $L_1^{(2)} \times L_2^{(2)}$  to construct the level-1 lists, and only the first  $k^{(1)}$  coordinates of the elements in level-1 lists satisfy this condition. Second, the  $k^{(2)}$  coordinates of the elements in level-2 lists are too small for effective hashing; using them would result in too many hash addresses or rounds. Instead, these  $k^{(2)}$  coordinates are later used to filter candidate pairs that matched on the remaining  $k^{(1)} - k^{(2)}$  coordinates. A similar strategy is used in step 5, where the hash function is applied to the  $m - k^{(1)}$  coordinates. However, here we omit the filtering step on the  $k^{(1)}$  coordinates because the full condition across all coordinates will be verified later in step 7. Finally, note that the hypothesis testing step (lines 6-8) involves two conditions. We will replace the second condition,  $\mathbf{A}\mathbf{x} - \mathbf{b} \bmod q \in [-E, E]^m$ , with a more efficient decoder later in our hybrid attack.

**Selection of  $(k^{(i)}, \ell^{(i)})$ .** The choice of parameters  $(k^{(i)}, \ell^{(i)})$  is crucial for successfully recovering the secret  $\mathbf{s}$ . The primary requirement is that *for each* element

<sup>6</sup> We note that the original construction for top-level lists in [20] follows a different MITM approach, which involves randomly sampling a subset of  $\mathcal{T}_0^n(\frac{w}{8})$ . Since Odlyzko's MITM is both simpler to implement and more efficient [31, Section 4], we adopt it in place of the original method in [20].

**Algorithm 2** Meet-LWE with Const.2 on solving Ternary-0 LWE

---

**Require:**  $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$  with  $\mathbf{s} \in \mathcal{T}_0^n(w)$  and  $\mathbf{e} \in [-E, E]^m$

**Ensure:**  $\mathbf{s} \in \mathcal{T}_0^n(w)$  satisfying  $\mathbf{A}\mathbf{s} - \mathbf{b} \bmod q \in [-E, E]^m$

- 1: compute parameters  $k^{(1)}, k^{(2)}$  and  $\ell^{(1)}, \ell^{(2)}$
- 2: construct level-3 lists as  $\mathcal{T}_0(w^{(3)})$  of size  $L^{(3)}$ 

$$L_i^{(3)} = \{(\mathbf{s}_i^{(3)}, \mathbf{A}\mathbf{s}_i^{(3)}) : \mathbf{s}_i^{(3)} \in \mathcal{T}_0^{n/2}(\frac{w}{8}) \times 0^{n/2}\}, i = 1, 3, 5, 7$$

$$L_i^{(3)} = \{(\mathbf{s}_i^{(3)}, \mathbf{b} - \mathbf{A}\mathbf{s}_i^{(3)}) : \mathbf{s}_i^{(3)} \in 0^{n/2} \times \mathcal{T}_0^{n/2}(\frac{w}{8})\}, i = 2, 4, 6$$

$$L_8^{(3)} = \{(\mathbf{s}_8^{(3)}, \mathbf{b} - \mathbf{A}\mathbf{s}_8^{(3)}) : \mathbf{s}_8^{(3)} \in 0^{n/2} \times \mathcal{T}_0^{n/2}(\frac{w}{8})\}$$
- 3: construct level-2 lists by performing hash function on the  $k^{(2)}$  coordinates of the second component of the elements in level-3 lists
$$L_i^{(2)} = \{(\mathbf{s}_i^{(2)}, \mathbf{A}\mathbf{s}_i^{(2)}) : \pi_{k^{(2)}}^m(\mathbf{A}\mathbf{s}_i^{(2)}) \in [-\ell^{(2)}, \ell^{(2)}]^{k^{(2)}}\} \text{ using } L_{2i-1}^{(3)} \times L_{2i}^{(3)}, i = 1, 2, 3$$

$$L_4^{(2)} = \{(\mathbf{s}_4^{(2)}, \mathbf{b} - \mathbf{A}\mathbf{s}_4^{(2)}) : \pi_{k^{(2)}}^m(\mathbf{b} - \mathbf{A}\mathbf{s}_4^{(2)}) \in [-\ell^{(2)}, \ell^{(2)}]^{k^{(2)}}\} \text{ using } L_7^{(3)} \times L_8^{(3)}$$
- 4: construct level-1 lists by first performing hash function on the  $k^{(1)} - k^{(2)}$  coordinates of the second component of elements in level-2 lists, and then filtering matched pairs based on the  $k^{(2)}$  coordinates
$$L_1^{(1)} = \{(\mathbf{s}_1^{(1)}, \mathbf{A}\mathbf{s}_1^{(1)}) : \pi_{k^{(1)}}^m(\mathbf{A}\mathbf{s}_1^{(1)}) \in [-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}}\} \text{ using } L_1^{(2)} \times L_2^{(2)}$$

$$L_2^{(1)} = \{(\mathbf{s}_2^{(1)}, \mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)}) : \pi_{k^{(1)}}^m(\mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)}) \in [-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}}\} \text{ using } L_3^{(2)} \times L_4^{(2)}$$
- 5: construct level-0 list  $L_1^{(0)}$  by performing hash function on the  $m - k^{(1)}$  coordinates of the second component of the elements in level-1 lists
- 6: **for** all matched  $(\mathbf{s}_1^{(1)}, \cdot)$  and  $(\mathbf{s}_2^{(1)}, \cdot)$  in  $L_1^{(0)}$  **do**
- 7:     **if**  $\mathbf{x} := \mathbf{s}_1^{(1)} + \mathbf{s}_2^{(1)} \in \mathcal{T}_0^n(w)$  and  $\mathbf{A}\mathbf{x} - \mathbf{b} \bmod q \in [-E, E]^m$  **then**
- 8:         **return**  $\mathbf{x}$

---

$\mathbf{s}_j^{(i-1)}$  at level- $(i-1)$ , there must exist at least one corresponding representation at level- $i$  with high probability. However, the analysis in [20] ensures this condition only *on average*, rather than for every individual  $\mathbf{s}_j^{(i-1)}$ . In what follows, we clarify this issue by carefully examining the conditions on the parameters  $(k^{(2)}, \ell^{(2)})$  and  $(k^{(1)}, \ell^{(1)})$ .

We begin with  $(k^{(2)}, \ell^{(2)})$ . To construct the level-1 list  $L_1^{(1)}$  (the analysis for  $L_2^{(1)}$  is analogous), we must select suitable parameters  $(k^{(2)}, \ell^{(2)})$  for the level-2 lists such that, for each element  $\mathbf{s}_1^{(1)}$  in  $L_1^{(1)}$ , at least one representation  $(\mathbf{s}_1^{(2)}, \mathbf{s}_2^{(2)})$  of  $\mathbf{s}_1^{(1)}$  survives in  $L_1^{(2)} \times L_2^{(2)}$  with high probability. Let  $p_R(\mathbf{s}_1^{(1)})$  denote the probability that a single representation of  $\mathbf{s}_1^{(1)}$  satisfies this condition. Since there are  $R^{(2)}$  representations, we require  $R^{(2)} \cdot p_R(\mathbf{s}_1^{(1)}) \geq 1$  to ensure at least one representation survives with probability  $1 - (1 - p_R(\mathbf{s}_1^{(1)}))^{R^{(2)}} \gtrsim 1 - \frac{1}{e}$ .

Define  $E_1(\mathbf{s}_1^{(1)})$  and  $E_2(\mathbf{s}_1^{(1)})$  as the events that the first  $k^{(2)}$  coordinates of  $\mathbf{A}\mathbf{s}_1^{(2)}$  and  $\mathbf{A}\mathbf{s}_2^{(2)}$ , respectively, lie within the box  $[-\ell^{(2)}, \ell^{(2)}]^{k^{(2)}}$ . Since  $\mathbf{A}\mathbf{s}_2^{(2)} = -\mathbf{A}\mathbf{s}_1^{(2)} + \mathbf{A}\mathbf{s}_1^{(1)}$ , we have

$$p_R(\mathbf{s}_1^{(1)}) = \underbrace{\Pr[E_1(\mathbf{s}_1^{(1)})]}_{=\left(\frac{2\ell^{(2)}}{q}\right)^{k^{(2)}}} \cdot \underbrace{\Pr[E_2(\mathbf{s}_1^{(1)}) | E_1(\mathbf{s}_1^{(1)})]}_{\text{conditional probability depends on } \mathbf{s}_1^{(1)}}.$$

Observe that the first term does not depend on  $\mathbf{s}_1^{(1)}$ , whereas the second conditional probability varies with the particular value of  $\mathbf{s}_1^{(1)}$ . In [20], this second term is approximated by its expectation over all  $\mathbf{s}_1^{(1)}$  in  $L_1^{(1)}$ :

$$p_{\text{exp}} = \mathbb{E}_{\mathbf{s}_1^{(1)} \in L_1^{(1)}} [\Pr[E_2(\mathbf{s}_1^{(1)}) | E_1(\mathbf{s}_1^{(1)})]] = (1 - \frac{\ell^{(1)}}{4\ell^{(2)}})^{k^{(2)}},$$

and then the parameters  $(k^{(2)}, \ell^{(2)})$  are chosen to satisfy

$$R^{(2)} \cdot (\frac{2\ell^{(2)}}{q})^{k^{(2)}} \cdot p_{\text{exp}} \gtrsim 1. \quad (6)$$

This approach is flawed. In practice, we must ensure that  $R^{(1)} \cdot p_R(\mathbf{s}_1^{(1)}) \gtrsim 1$  holds for *every*  $\mathbf{s}_1^{(1)}$  in  $L_1^{(1)}$ , not merely on average. If the parameters  $(k^{(2)}, \ell^{(2)})$  are chosen according to condition 6, then for elements  $\mathbf{s}_1^{(1)}$  with below-average conditional probability, the expected number of surviving representations in  $L_1^{(2)} \times L_2^{(2)}$  will fall below one, resulting in missing elements in  $L_1^{(1)}$ . Conversely, elements with above-average probabilities yield extra representations, but only one is retained in  $L_1^{(1)}$ . As a result, these additional representations cannot compensate for the missing ones, and the overall correctness of the construction is not preserved.

This limitation is acknowledged in [20]. To mitigate it, [20] adopts a more conservative strategy: the parameters  $(k^{(2)}, \ell^{(2)})$  are chosen such that  $R^{(2)} \cdot (\frac{2\ell^{(2)}}{q})^{k^{(2)}} \cdot p_{\text{exp}} \gtrsim 10$ , which is intended to ensure that the survival probability is at least  $1 - \frac{1}{e^{10}}$ . Subsequently, they conduct experiments to estimate the actual expected probability that an element  $\mathbf{s}_1^{(1)}$  in  $L_1^{(1)}$  has at least one surviving representation. Their empirical results show that this probability remains above 0.6 when  $k^{(2)} \leq 100$ , and it could be smaller for larger dimension.

To enhance the general applicability of the method and ground it in a more rigorous theoretical framework, we propose a different and simpler approach for parameter selection. Rather than relying on the average-case behavior, we analyze the worst-case conditional probability, which occurs when the first  $k^{(1)}$  coordinates of  $\mathbf{A}\mathbf{s}_1^{(1)}$  are all equal to  $\ell^{(1)}$ :

$$\begin{aligned} p_{\min} &= \min_{\mathbf{s}_1^{(1)} \in L_1^{(1)}} \Pr[E_2(\mathbf{s}_1^{(1)}) | E_1(\mathbf{s}_1^{(1)})] \\ &= (\Pr[-x + \ell^{(1)} \in [-\ell^{(2)}, \ell^{(2)}] | x \in [-\ell^{(2)}, \ell^{(2)}]])^{k^{(2)}} = (1 - \frac{\ell^{(1)}}{2\ell^{(2)}})^{k^{(2)}}. \end{aligned}$$

We then adopt this worst-case bound to guide parameter selection and require

$$R^{(2)} \cdot (\frac{2\ell^{(2)}}{q})^{k^{(2)}} \cdot (1 - \frac{\ell^{(1)}}{2\ell^{(2)}})^{k^{(2)}} \gtrsim 1. \quad (7)$$

This condition guarantees that every element  $\mathbf{s}_1^{(1)}$  in  $L_1^{(1)}$  has at least one surviving representation with high probability. Moreover, condition 7 implies a natural constraint on adjacent levels, namely,  $\ell^{(1)} < 2\ell^{(2)}$  must hold.

We now turn to the condition on  $(k^{(1)}, \ell^{(1)})$ , where the analysis is consistent with [20]. In this case, it suffices to ensure that, for the target secret  $\mathbf{s}$ , at least one representation  $(\mathbf{s}_1^{(1)}, \mathbf{s}_2^{(1)})$  survives in  $L_1^{(1)} \times L_2^{(1)}$  with high probability. Unlike the previous case, here we only need to consider one fixed target  $\mathbf{s}$ . Let  $p_R(\mathbf{s})$  denote the probability that a single representation of  $\mathbf{s}$  satisfies this condition. Define  $E_1(\mathbf{s})$  and  $E_2(\mathbf{s})$  as the events that the first  $k^{(1)}$  coordinates of  $\mathbf{A}\mathbf{s}_1^{(1)}$  and  $\mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)}$ , respectively, lie within the box  $[-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}}$ . Note that  $\mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)} = \mathbf{A}\mathbf{s}_1^{(1)} + \mathbf{e}$ . Assuming independence across coordinates and denoting by  $e$  a single component of the error vector  $\mathbf{e}$ , we have

$$\begin{aligned} p_R(\mathbf{s}) &= \mathbb{E}_{\mathbf{e} \in \chi^m} (\Pr[E_1(\mathbf{s})] \cdot \Pr[E_2(\mathbf{s})|E_1(\mathbf{s})]) = \left(\frac{2\ell^{(1)}}{q}\right)^{k^{(1)}} \cdot \mathbb{E}_{\mathbf{e} \in \chi^m} \Pr[E_2(\mathbf{s})|E_1(\mathbf{s})] \\ &= \left(\frac{2\ell^{(1)}}{q}\right)^{k^{(1)}} \cdot (\mathbb{E}_{e \in \chi} \Pr[x + e \in [-\ell^{(1)}, \ell^{(1)}] | x \in [-\ell^{(1)}, \ell^{(1)}]])^{k^{(1)}} \\ &= \left(\frac{2\ell^{(1)}}{q}\right)^{k^{(1)}} \cdot p_c(E, \ell^{(1)})^{k^{(1)}}, \end{aligned}$$

where  $p_c(E, \ell^{(1)})$  is defined in Lemma 1. Therefore,  $(k^{(1)}, \ell^{(1)})$  satisfies

$$R^{(1)} \cdot \left(\frac{2\ell^{(1)}}{q}\right)^{k^{(1)}} \cdot p_c(E, \ell^{(1)})^{k^{(1)}} \gtrsim 1. \quad (8)$$

Given the above conditions, we determine the optimal values for  $(k^{(i)}, \ell^{(i)})$  by performing an exhaustive search over all  $0 \leq k^{(2)} \leq k^{(1)} \leq m$ . For each pair  $(k^{(1)}, k^{(2)})$ , we compute the corresponding values of  $\ell^{(1)}$  and  $\ell^{(2)}$  based on conditions (7) and (8), respectively, ensuring that the constraint  $\ell^{(2)} < 2\ell^{(1)}$  is satisfied.

**Time analysis.** The sizes of the lists at each level are given by  $L^{(3)} = |\mathcal{T}_0^{n/2}(\frac{w}{8})|$ ,  $L^{(2)} = |\mathcal{T}_0^n(\frac{w}{4})| \cdot \left(\frac{2\ell^{(2)}}{q}\right)^{k^{(2)}}$ ,  $L^{(1)} = |\mathcal{T}_0^n(\frac{w}{2})| \cdot \left(\frac{2\ell^{(1)}}{q}\right)^{k^{(1)}}$ . The size of  $L^{(0)}$  depends on the hash function from level-1 lists to level-0 list (see the explanation of Eq.(3) and Eq.(4) for the number of collisions of using Hash1 and Hash2, respectively). Denote  $T^{(i)}$  the time of performing hash function on level- $i$  lists to generate level- $(i-1)$  lists ( $i = 1, 2, 3$ ). The time of Hash1 and Hash2 are given in Eq.(3) and Eq.(4), respectively. Then the overall time of Const.2 is

$$T_{\text{Const.2}} = \max \left\{ L^{(3)}, L^{(2)}, L^{(1)}, L^{(0)}, T^{(3)}, T^{(2)}, T^{(1)} \right\}. \quad (9)$$

## 4 Comparison of Meet-LWE Variants

In the previous section, we revisited two recently proposed techniques aimed at addressing the primary bottlenecks of Meet-LWE, namely, error enumeration and hash label explosion. While both techniques are designed to improve the efficiency of Meet-LWE, their actual effectiveness—whether they offer consistent

gains and under what conditions—remains incompletely understood, particularly within the pure MITM setting. In this section, we conduct a systematic and in-depth comparison of these techniques, both individually and in combination, within the Meet-LWE framework. Our analysis reveals that their performance exhibits subtler and more complex behavior than initially expected. In particular, we highlight the following insights.

- **List Construction (Const.2):** Somewhat surprisingly, the new list construction (Const.2) often results in worse performance across a wide range of parameter settings. This is due to a subtle but fundamental structural bottleneck,<sup>7</sup> which we analyze in detail.
- **Hash Function (Hash2):** As expected, the new hash function Hash2 consistently outperforms Hash1, with its advantage becoming more pronounced when the error distribution is large.
- **Combined Use:** When used together, Hash2 and Const.2 complement each other: Hash2 mitigates the primary bottleneck of Const.2, while Const.2 amplifies the benefits of Hash2. This combination achieves better overall performance in most cases, though there remain parameter regimes where further improvements are needed.

We hope these insights clarify the nuanced behavior of these techniques and also provide a foundation for future improvements to Meet-LWE and its variants.

To facilitate a structured comparison, we define four Meet-LWE variants based on the choice of list construction and hash function: MITM1 is the original Meet-LWE; H1C2 replaces Const.1 by Const.2; H2C1 replaces Hash1 by Hash2, and MITM2 combines both changes. Since these attacks involve many interdependent parameters and exhibit complex interactions, we begin in Section 4.1 by analyzing the dependencies among key parameters. Building on this, Section 4.2 compares the four variants under three representative parameter settings, providing insights into their relative strengths and limitations. Finally, Section 4.3 offers a broader comparison across a wide parameter range.

#### 4.1 Parameter Dependency

We begin with several observations regarding the dependency among parameters under Const.2. Recall that parameters  $(k^{(i)}, \ell^{(i)})$  must satisfy the following conditions (7) and (8):

$$R^{(2)} \cdot \left(\frac{2\ell^{(2)}}{q}\right)^{k^{(2)}} \cdot \left(1 - \frac{\ell^{(1)}}{2\ell^{(2)}}\right)^{k^{(2)}} \gtrsim 1 \text{ and } R^{(1)} \cdot \left(\frac{2\ell^{(1)}}{q}\right)^{k^{(1)}} \cdot p_c(E, \ell^{(1)})^{k^{(1)}} \gtrsim 1.$$

From these inequalities, we observe that  $(k^{(i)}, \ell^{(i)})$  must vary in the same direction in order to maintain feasibility.

**Observation 1.** *Under Const.2, for each level  $i = 1, 2$ , increasing  $k^{(i)}$  requires increasing  $\ell^{(i)}$ , and vice versa, assuming other parameters remain fixed.*

<sup>7</sup> This limitation persists even with the parameter selection strategy proposed in [20].

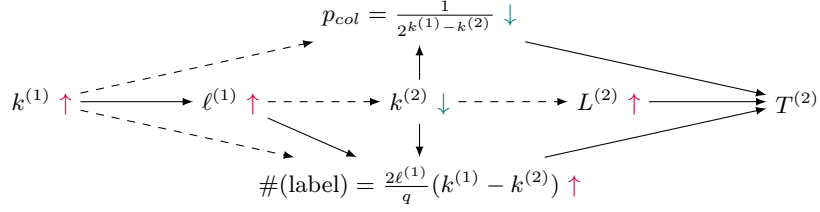


Fig. 1: Parameter dependencies under H1C2 and the effect of increasing  $k^{(1)}$ . An arrow “ $\rightarrow$ ” (resp. “ $--\rightarrow$ ”) from node  $A$  to node  $B$  indicates that increasing  $A$  cause  $B$  to increase (resp. decrease).

Furthermore, conditions 7 and (8) imply that  $(\frac{2^{\ell^{(i)}}}{q})^{k^{(i)}} \geq \frac{1}{R^{(i)}}$ . Hence, the list size under Const.2 satisfies  $L^{(i)} = |\mathcal{T}_0^n(\frac{w}{2^i})| \cdot (\frac{2^{\ell^{(i)}}}{q})^{k^{(i)}} \geq |\mathcal{T}_0^n(\frac{w}{2^i})| \cdot \frac{1}{R^{(i)}}$ , where the right-hand side corresponds to the the list size under Const.1.

**Observation 2.** *For each level  $i = 1, 2$ , the list size under Const.2 is larger than that under Const.1.*

Finally, from condition 7 for  $(k^{(2)}, \ell^{(2)})$ , we can see that increasing  $\ell^{(1)}$  will decrease the last term  $(1 - \frac{\ell^{(1)}}{2^{\ell^{(2)}}})^{k^{(2)}}$ . To compensate, one must decrease  $k^{(2)}$  to increase the middle term  $(\frac{2^{\ell^{(2)}}}{q})^{k^{(2)}}$ , which will then increase the list size  $L^{(2)}$ .

**Observation 3.** *Under Const.2, increasing  $\ell^{(1)}$  requires decreasing  $k^{(2)}$ , which in turn increases the level-2 list size  $L^{(2)}$ .*

The above dependencies among parameters are summarized in Fig.1 and will be used in the next subsection.

## 4.2 Comparison on Three Typical Schemes

We compare the performance of the four Meet-LWE variants using three representative parameter sets from sparse ternary LWE. The first scheme “NTRU Prime 653” was originally used in [31] to illustrate the effect of MITM1. The second scheme “FHE 1024” features a much larger  $q$  than “NTRU Prime 653”. The third scheme “FHE 1024 large error” is representative of hybrid dual attack scenarios, in which the LWE instance resulting from the reduction stage typically exhibits a large error. Table 1 summarizes the parameters of these schemes and the corresponding bit-security estimates obtained using each attack variant.

**Const.1 vs Const.2.** We first compare Const.1 and Const.2 using the results of MITM1 and H1C2. Using Const.2 avoids the error-guessing time  $T_e$ , but at the cost of increasing the list size  $L^{(1)}$  (Obs. 2), which subsequently increases  $T^{(1)}$ . For all three schemes, this trade-off is favorable, as  $T^{(1)}$  under H1C2 is always smaller than the combined cost  $T^{(1)} + T_e$  under MITM1. However, this local

advantage does not always translate into a global one. In fact, for the first two schemes, the overall time complexity under H1C2 is surprisingly larger than that under MITM1. This demonstrates that adopting Const.2 does not necessarily yield better performance. The root cause is that the computational bottleneck under H1C2 lies in  $T^{(2)}$ , which dominates  $T^{(1)}$  in these cases.

Recall that  $T^{(2)}$  is the time to apply Hash1 to the first  $k^{(1)} - k^{(2)}$  coordinates of the elements in level-2 lists in order to find  $\ell^{(1)}$ -close pairs:

$$\begin{aligned} T^{(2)}(\text{H1C2}) &= \max\{L^{(2)} \cdot \#(\text{label}), (L^{(2)} \cdot \#(\text{label}))^2 \cdot \frac{1}{2^{k^{(1)} - k^{(2)}}}\} \\ &= (L^{(2)} \cdot \#(\text{label}))^2 \cdot \frac{1}{2^{k^{(1)} - k^{(2)}}}, \end{aligned}$$

where the first term is the time to computing hash labels and the second to find collisions. The second term dominates for all three schemes because  $k^{(1)} - k^{(2)}$  is small, resulting in a high collision probability.

Since  $T^{(2)}$  is large for the first two schemes, one natural idea is to increase the gap  $k^{(1)} - k^{(2)}$  to reduce the collision cost. However, this turns out to be ineffective due to a chain reaction among the parameters. As shown in Fig.1, increasing  $k^{(1)}$  also increases  $\ell^{(1)}$  (Obs. 1), which then leads to a smaller  $k^{(2)}$  and a larger  $L^{(2)}$  (Obs. 3). Although this indeed widens the gap  $k^{(1)} - k^{(2)}$ , which reduces the collision probability, it also increases both the number of labels and the list size  $L^{(2)}$ . These effects outweigh the gain, and as a result, increasing  $k^{(1)}$  can actually worsen  $T^{(2)}$ . In fact, the current parameter settings for the first two schemes already achieve the optimal trade-off for minimizing  $T^{(2)}$ .

Another idea to reduce  $T^{(2)}$  is to introduce additional levels of lists. However, this also does not help as adding more levels primarily reduces the time for constructing the level-2 lists (i.e.,  $T^{(3)}$ ), which is already smaller than the bottleneck  $T^{(2)}$ . Therefore, it does not alleviate the overall runtime.

**Hash1 vs Hash2.** We then compare Hash1 and Hash2 by examining the performance of MITM1 and H2C1. Both attacks share the same list construction (Const.1), and thus the error enumeration cost  $T_e$  is identical in both. Their total runtime is given by  $T(\text{MITM1}/\text{H2C1}) = \max\{T^{(1)}, T^{(2)}\} + T_e = T^{(1)} + T_e$ , since for these all schemes,  $T^{(1)} \geq T^{(2)}$ . The key difference lies in  $T^{(1)}$ , the time of applying hash function on level-1 lists to find collisions. Hash1 assigns multiple hash labels to each elements in  $L_1^{(1)}$  and  $L_2^{(1)}$ , whereas Hash2 avoids this redundancy but requires multiple rounds of hashing. Therefore, we can rewrite  $T^{(1)}$  for each method as

$$T^{(1)}(\text{MITM1}) = L^{(1)} \cdot \#(\text{label}) \quad \text{and} \quad T^{(1)}(\text{H2C1}) = L^{(1)} \cdot \#(\text{round}).$$

Both the number of hash labels and the number of rounds depend primarily on the ratio  $\frac{E}{q}$ . As this ratio is approaching 0 for the first two schemes, both terms are less than 1 bit, so the runtimes of MITM1 and H2C1 are nearly identical. In contrast, for the third scheme, the ratio is  $\frac{E}{q} = 0.02$ , leading to  $\#(\text{label}) = 2^{24}$  and  $\#(\text{round}) = 2^6$ . This difference directly explains the gap

Table 1: Comparison of  $\log_2$ -based complexity for four variants of Meet-LWE

| Scheme                        | Attack | Cost (bit) |           |           |           |       | List size (bit) |           |           | Parameters |           |              |              |
|-------------------------------|--------|------------|-----------|-----------|-----------|-------|-----------------|-----------|-----------|------------|-----------|--------------|--------------|
|                               |        | $T$        | $T^{(1)}$ | $T^{(2)}$ | $T^{(3)}$ | $T_e$ | $L^{(1)}$       | $L^{(2)}$ | $L^{(3)}$ | $k^{(1)}$  | $k^{(2)}$ | $\ell^{(1)}$ | $\ell^{(2)}$ |
| NTRU<br>Prime<br>653          | MITM1  | 370        | 352       | 352       | -         | 18    | 352             | 313       | -         | 23         | -         | -            | -            |
|                               | H2C1   | 370        | 352       | 352       | -         | 18    | 352             | 313       | -         | 23         | -         | -            | -            |
|                               | H1C2   | 507        | 367       | 507       | 365       | -     | 367             | 256       | 192       | 26         | 21        | 2            | 27           |
|                               | MITM2  | 402        | 357       | 402       | 268       | -     | 357             | 274       | 192       | 30         | 14        | 4            | 7            |
| FHE<br>1024                   | MITM1  | 180        | 173       | 173       | -         | 7     | 173             | 115       | -         | 3          | -         | -            | -            |
|                               | H2C1   | 180        | 173       | 173       | -         | 7     | 173             | 115       | -         | 3          | -         | -            | -            |
|                               | H1C2   | 204        | 173       | 204       | 126       | -     | 173             | 102       | 64        | 4          | 3         | 16           | 638          |
|                               | MITM2  | 172        | 172       | 172       | 123       | -     | 172             | 128       | 64        | 87         | 5         | 323334       | 332534       |
| FHE<br>1024<br>large<br>error | MITM1  | 222        | 197       | 173       | -         | 25    | 173             | 115       | -         | 3          | -         | -            | -            |
|                               | H2C1   | 204        | 179       | 173       | -         | 25    | 173             | 115       | -         | 3          | -         | -            | -            |
|                               | H1C2   | 212        | 202       | 212       | 123       | -     | 178             | 108       | 64        | 14         | 10        | 34369        | 103820       |
|                               | MITM2  | 180        | 180       | 180       | 124       | -     | 175             | 128       | 64        | 64         | 4         | 280142       | 283491       |

\* NTRU Prime 653:  $(n, q, w) = (653, 4621, 288)$  and error  $\mathbf{e} \leftarrow \mathcal{U}(\{0, \pm 1\}^m)$ .

\* FHE 1024:  $(\log n, \log q, w) = (10, 20, 64)$  and error  $\mathbf{e} \leftarrow \mathcal{G}_{3.2}$ .

\* FHE 1024 large error:  $(\log n, \log q, w) = (10, 20, 64)$  and error  $\mathbf{e} \leftarrow \mathcal{G}_{0.02*q}$ .

between the runtimes:  $(222 - 204 = 18 = 24 - 6)$ . In summary, the advantage of Hash2 is more pronounced when the ratio  $\frac{E}{q}$  is large, which is exactly the situation encountered in hybrid attacks. Thus, Hash2 provides greater benefits for MITM in a hybrid framework than used alone.

**Combining Hash2 and Const.2.** Finally we compare MITM1 and MITM2 to see the benefits of combining Hash2 and Const.2. MITM2 outperforms MITM1 for the last two schemes, while it performs worse on the first one. The most intriguing case is the second scheme, where using Hash2 along brings no gain, and Const.2 alone increases complexity, but combining both reduces the overall complexity by 8 bits. We analyze this case in more detail below.

As discussed in “Const.1 vs Const.2”, the main obstacle to using Const.2 in H1C2 is the large  $T^{(2)}$ , and attempts to reduce it by increasing  $k^{(1)}$  are ineffective: while a larger  $k^{(1)}$  reduces the collision probability, it also increases both the list size  $L^{(2)}$  and the number of labels, negating the benefit. Now consider what happens when we additionally replace Hash1 with Hash2, resulting in MITM2. This brings two key advantages. First, the large number of labels is be replaced by a much smaller number of hashing rounds (as explained in “Hash1 vs Hash2”). Notably, from the perspective of Hash2, Const.2 amplifies its advantage because the number of labels under Const.2 is large, making Hash2 especially effective. Second, and more importantly, Hash2’s ability to partition the torus into multiple segments greatly reduces the collision probability. This enables more aggressive use of a larger  $k^{(1)}$ : while  $L^{(2)}$  still increases, the reduction in collision probability dominates, resulting in an overall decrease in  $T^{(2)}$ . To summarize, combining Hash2 and Const.2 is strictly more powerful

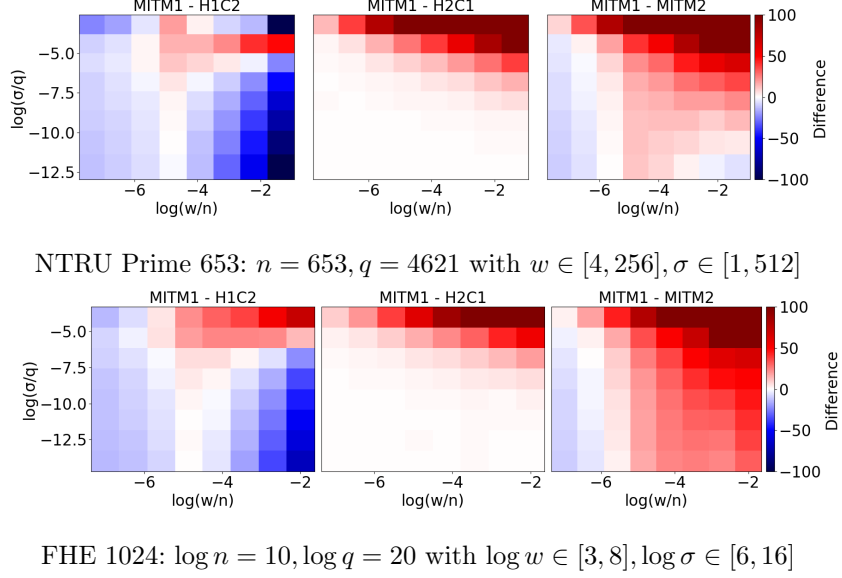


Fig. 2: Comparison of Meet-LWE variants across a wide range of parameters. Red regions indicate instances where the new techniques outperform the original MITM1, with darker shades representing greater performance gains.

than using either one alone, since Hash2 eliminates the primary bottleneck of Const.2, and Const.2 amplifies the efficiency gains brought by Hash2.<sup>8</sup>

### 4.3 Comparison on A Large Range of Parameters

In addition to the three schemes discussed earlier, we extend our comparison across a broader range of parameters. In particular, we fix two  $(n, q)$  settings corresponding to the first two schemes in Table 1 and vary the Hamming weight  $w$  and error standard deviation  $\sigma$ . The parameter settings and results are summarized in Fig.2.

The trends in Fig.2 align with the conclusions drawn in the previous subsection. The two leftmost plots show that Const.2 often underperforms compared to Const.1 (blue regions), especially when  $w$  is large. The two middle plots demonstrate that Hash2 consistently outperforms Hash1, with its advantage growing as  $\sigma$  and  $w$  increase. The rightmost plots, comparing MITM1 and MITM2, reveal a more nuanced picture. Both plots show that MITM2 performs worse when  $w$  is small. In the top-right plot, MITM2 also underperforms in the bottom-right region, which includes the “NTRU Prime 653” parameters.

<sup>8</sup> This synergy is also observable in the first scheme, where MITM2 outperforms H1C2, although it still does not outperform MITM1.

These findings indicate that combining Const.2 and Hash2 offers improvements in most parameter settings, particularly when the error variance is large, as is typical in hybrid dual attack settings. This motivates the new hybrid dual attack introduced in the next section. That said, there remain cases where the combined method performs worse, highlighting an open challenge for future work.

## 5 Our New Hybrid Dual Attack

In this section we present our new hybrid dual attack by combining dual attack and MITM2 defined in Section 3. As usual, our new attack has two stages: reduction-stage and guessing-stage. The reduction-stage is the same as other hybrid dual attacks, which outputs a new instance with a smaller dimension, a larger error, and also a different secret distribution. Then in the guessing-stage we apply MITM2 with the following two modifications to solve this new instance. In order to cope with the different secret distribution, we analyze the choice of the parameters for the list construction and the corresponding success probability. For the larger error, we replace the original trivial distinguish method in the last hypothesis testing step of MITM2 by a better decoder from Laarhoven-Walter [25]. We present the reduction-stage in Section 5.1 and the guessing-stage highlighting the two modifications in Section 5.2, and then summarize our algorithm and its complexity in Section 5.3.

### 5.1 Reduction-stage [5]

Given an instance  $(\mathbf{A}, \mathbf{b}) \leftarrow \text{LWE}_{n,q,\mathbf{s},\mathcal{G}_\sigma}$  with  $\mathbf{s} \in \mathcal{T}_0^n(w)$ , we divide the matrix  $\mathbf{A}$  and secret  $\mathbf{s}$  into two parts:  $\mathbf{A} = (\mathbf{A}_1, \mathbf{A}_2) \in \mathbb{Z}_q^{m \times r} \times \mathbb{Z}_q^{m \times (n-r)}$ ,  $\mathbf{s} = (\mathbf{s}_1, \mathbf{s}_2) \in \{0, \pm 1\}^r \times \{0, \pm 1\}^{n-r}$ . Then we construct the dual lattice over  $\mathbf{A}_2$ :  $\Lambda^\perp(\mathbf{A}_2) = \{(\mathbf{w}, \mathbf{v}) \in \mathbb{Z}^m \times (\frac{1}{c} \cdot \mathbb{Z})^{n-r} : \mathbf{w} \cdot \mathbf{A}_2 = c \cdot \mathbf{v} \bmod q\}$ , where  $c$  is the scale factor and  $m = \sqrt{\frac{(n-r) \cdot \log(q/c)}{\log \delta_0}}$  [32], which depends on  $\delta_0$  (defined in Section 2.2). For a short vector  $(\mathbf{w}, \mathbf{v}) \in \Lambda^\perp(\mathbf{A}_2)$ , we compute

$$\begin{aligned} \langle \mathbf{w}, \mathbf{b} \rangle &= \mathbf{w}(\mathbf{A}\mathbf{s} + \mathbf{e}) = \mathbf{w}\mathbf{A}_1\mathbf{s}_1 + \mathbf{w}\mathbf{A}_2\mathbf{s}_2 + \langle \mathbf{w}, \mathbf{e} \rangle \\ &= \mathbf{w}\mathbf{A}_1\mathbf{s}_1 + c \cdot \langle \mathbf{v}, \mathbf{s}_2 \rangle + \langle \mathbf{w}, \mathbf{e} \rangle \bmod q. \end{aligned}$$

This can be seen as a new LWE instance  $(\bar{\mathbf{a}}, \bar{\mathbf{b}} = \langle \bar{\mathbf{a}}, \mathbf{s}_1 \rangle + \bar{\mathbf{e}} \bmod q)$  with

$$\begin{aligned} \bar{\mathbf{b}} &= \langle \mathbf{w}, \mathbf{b} \rangle \bmod q, \\ \bar{\mathbf{a}} &= \mathbf{w}\mathbf{A}_1 \bmod q, \\ \bar{\mathbf{e}} &= c \cdot \langle \mathbf{v}, \mathbf{s}_2 \rangle + \langle \mathbf{w}, \mathbf{e} \rangle \bmod q. \end{aligned} \tag{10}$$

By using  $M$  short vectors  $\mathcal{W} = \{(\mathbf{w}, \mathbf{v})_1, \dots, (\mathbf{w}, \mathbf{v})_M\}$ , we obtain a set of new LWE instance

$$\bar{\mathbf{A}}\mathbf{s}_1 + \bar{\mathbf{e}} = \bar{\mathbf{b}} \bmod q, \tag{11}$$

where  $\bar{\mathbf{b}}, \bar{\mathbf{e}} \in \mathbb{Z}_q^M$  and  $\bar{\mathbf{A}} \in \mathbb{Z}_q^{M \times r}$ . This instance follows distribution  $\text{LWE}_{r,q,\mathbf{s}_1,\mathcal{G}_\rho}$  with  $\rho = \ell\sigma$  where  $\ell = \|(\mathbf{w}, \mathbf{v})\|$  [4]. Notice that this new instance has a special secret distribution as  $\mathbf{s}_1$  is part of the original secret  $\mathbf{s}$ . Denote  $hm(\mathbf{s}_1)$  the Hamming weight of  $\mathbf{s}_1$  and  $w_r := \min(r, w)$ , then we have  $hm(\mathbf{s}_1) \in [0, w_r]$ .

*Remark 1.* Note that the error term  $\bar{\mathbf{e}}$  contains the part secret  $\mathbf{s}_2$ . Based on the subsequent sections,  $hm(\mathbf{s}_2) \in [w_r, w]$ , and for the purpose of the calculation of the bit-security, we assume a fixed  $hm(\mathbf{s}_2) = w$ , which gives an upper bound on the bit-security estimation of our attack.

## 5.2 Guessing-stage

Next we solve the above new LWE instance by using MITM2. First we select a Hamming weight parameter  $\hat{w} \in (0, w_r]$  to determine which candidate solutions for  $\mathbf{s}_1$  can be solved by MITM2. Let  $\hat{w}$  be even and we only solve those candidate solutions with  $\#(1) = \#(-1)$ , i.e.,  $\mathbf{s}_1 \leftarrow \mathcal{U}_1^r(\hat{w}, p_s(h))$  with

$$p_s(h) := \binom{r}{h, h} \binom{n-r}{(r-2h)/2, (r-2h)/2} / \binom{n}{w/2, w/2}, \quad (12)$$

to be the probability for  $hm(\mathbf{s}_1) = 2h$  with  $0 \leq 2h \leq \hat{w}$ . Then the probability of forming the target secret  $\mathbf{s}_1$  is  $\sum_{h=0}^{\hat{w}/2} p_s(h)$ .

*Remark 2.* Note that in the hybrid setting we do not need to consider the representation technique as in [31] here, since the Hamming weight parameter  $\hat{w}$  is chosen by ourselves, and adding additional Hamming weight is equivalent to choosing a larger  $\hat{w}$ . We will search for the optimal  $\hat{w} \in (0, w_r]$  that minimizes the overall attack complexity.

**List Construction.** The main difference in the list construction for LWE with the special secret distribution is the choice of parameters  $(k^{(1)}, \ell^{(1)})$  for level-1 lists. The upper levels of lists construction here are the same as that in Section 3 and we will not dwell on it here.

Recall that for normal secret distribution (see Section 3), these parameters are chosen such that at least one representation of the secret survives with high probability in the lists. Now we need to fix one pair of parameters for all candidates with different Hamming weights, which will introduce an additional success probability. More precisely, for each  $h \leq \frac{\hat{w}}{2}$ , the representation number of  $\mathbf{s}_1 = \mathbf{s}_1^{(1)} + \mathbf{s}_2^{(1)}$  with  $hm(\mathbf{s}_1) = 2h$  is  $R^{(1)}(h) = \binom{r-2h}{\hat{w}/2-h} \sum_{k=0}^{\hat{w}/2-h} \binom{\hat{w}/2-h}{k} \binom{h}{\hat{w}/4-k}$ , and the probability for  $\pi_{k^{(1)}}^M(\mathbf{A}\mathbf{s}_1^{(1)}) \in [-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}}$  and  $\pi_{k^{(1)}}^M(\mathbf{b} - \mathbf{A}\mathbf{s}_2^{(1)}) \in [-\ell^{(1)}, \ell^{(1)}]^{k^{(1)}}$  can be computed as

$$p_\pi(h) := 1 - \left(1 - \left(\frac{2\ell^{(1)}}{q} \cdot p_c(\rho, 2\ell^{(1)})\right)^{k^{(1)}}\right)^{R^{(1)}(h)}. \quad (13)$$

Therefore the overall success probability is  $\sum_{h=0}^{\hat{w}/2} p_s(h) \cdot p_\pi(h)$ .

**Hypothesis Testing.** Next we improve the efficiency of MITM2 in the setting of larger error by replacing the condition  $\mathbf{Ax} - \mathbf{b} \bmod q \in [-E, E]^m$  of line 6 in Algorithm 2 by a different decoder. Among the set of candidate pairs  $(\tilde{\mathbf{s}}_1^{(1)}, \tilde{\mathbf{s}}_2^{(1)})$  (i.e., the line 5 in Algorithm 2), one of them can form the correct secret  $\mathbf{s}_1$ , which satisfies  $\tilde{\mathbf{t}} := \tilde{\mathbf{b}} - \tilde{\mathbf{A}}(\mathbf{s}_1^{(1)} + \mathbf{s}_2^{(1)}) = \tilde{\mathbf{b}} - \tilde{\mathbf{A}}\mathbf{s}_1 = \tilde{\mathbf{e}}$ , and hence follows the Gaussian distribution  $\mathcal{G}_\rho$  defined as Eq.(11). The remaining vectors  $\tilde{\mathbf{t}}$  associated with wrong pairs are uniform in  $\mathbb{Z}_q^M$ . This is different from applying MITM2 on LWE with sparse ternary secrets, where we can simply check whether the resulting vectors are in the range  $[-1, 1]^M$ . Although we can still choose a parameter, say  $E = 3\rho$ , and find the correct candidate by checking whether  $\tilde{\mathbf{t}} \in [-E, E]^M$ , this would require a larger  $M$  to maintain a reasonable success probability as the ratio  $\frac{E}{q}$  is much larger than  $\frac{1}{q}$ .

In order to improve the efficiency, we use the distinguisher referred to as the simple-decoder [25] in the hypothesis testing. For all candidates  $\tilde{\mathbf{s}}_1 := \tilde{\mathbf{s}}_1^{(1)} + \tilde{\mathbf{s}}_2^{(1)}$ , we first check whether it satisfies the hamming weight condition of correct  $\mathbf{s}_1$  and then compute the score of them by using the simple-decoder and output the one with the highest score. The score for each candidate is given by  $f(\tilde{\mathbf{s}}_1) = \sum_{i \in [M]} \cos(\frac{2\pi}{q} \cdot \tilde{\mathbf{t}}_i)$ . The score can be modelled as a Gaussian of center  $M \cdot \mathbb{E}$  and variance  $M \cdot \mathbb{V}$ , where  $\mathbb{E}, \mathbb{V}$  depend on the accuracy of guessing. When guessing correctly,  $\mathbb{E}_G \approx \epsilon, \mathbb{V}_G \approx \frac{1}{2} - \Theta(\epsilon^2)$ , where  $\epsilon = \exp(\frac{-2\pi^2 \rho^2}{q^2})$ , and they are  $\mathbb{E}_U = 0, \mathbb{V}_U = \frac{1}{2}$  when guessing wrongly, according to [25]. Following [13, 17, 30], we denote the candidate number of hypothesis testing as  $T$ , and then the probability of success of using

$$M \geq \frac{1}{\epsilon^2} \cdot \ln T \quad (14)$$

short vectors to distinguish all these candidates is

$$p_t(M) = 1 - \frac{T}{2} \cdot \operatorname{erfc}(\sqrt{M}\epsilon). \quad (15)$$

*Remark 3.* When using the same number of short vectors, the simple-decoder has better success probability than the trivial method used in MITM and previous hybrid dual attacks focused on FHE (e.g., [5, 10]). The detailed analysis of the trivial distinguishing method can be found in [5, 10].

### 5.3 Our New Attack

Our attack is summarized in Algorithm 3. We first construct a new LWE instance  $(\tilde{\mathbf{A}}, \tilde{\mathbf{b}})$  as Section 5.1. Then we solve the new instance using the adjusted MITM2 in Section 5.2. According to the analysis above, we summarize the runtime of our new hybrid dual attack in Theorem 1.

**Theorem 1.** *Given an LWE instance  $(\mathbf{A}, \mathbf{b} = \mathbf{As} + \mathbf{e} \bmod q) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$  with  $\mathbf{s} \leftarrow \mathcal{U}_0^n(w)$  and  $\mathbf{e} \leftarrow \mathcal{G}_\sigma$ , the runtime of our new hybrid dual attack on*

**Algorithm 3** New Hybrid Dual Attack**Require:**  $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m, r, \beta, \sigma, \hat{w}, M, (k^{(1)}, \ell^{(1)}), (k^{(2)}, \ell^{(2)})$ **Ensure:**  $\mathbf{s}_1 \in \mathcal{T}_1^r(w_r)$ 

- 1: Divide  $\mathbf{A}$  into two parts  $(\mathbf{A}_1, \mathbf{A}_2) \in \mathbb{Z}_q^{m \times r} \times \mathbb{Z}_q^{m \times (n-r)}$
- 2: Construct lattice  $\Lambda^\perp(\mathbf{A}_2) = \{(\mathbf{w}, \mathbf{v}) \in \mathbb{Z}^m \times \mathbb{Z}^{n-r} : \mathbf{w} \cdot \mathbf{A}_2 = \mathbf{v} \bmod q\}$  and perform BKZ and sieving with blocksize  $\beta$  on it to obtain  $M$  short vectors  $(\mathbf{w}, \mathbf{v})$  of length  $\ell$  (see Section 2.2)
- 3: Construct new instance  $(\bar{\mathbf{A}}, \bar{\mathbf{b}}) \in \mathbb{Z}_q^{M \times r} \times \mathbb{Z}_q^M$  by computing each row/entry of  $\bar{\mathbf{A}}$  and  $\bar{\mathbf{b}}$  as  $\bar{\mathbf{a}} = \mathbf{w}\mathbf{A}_1 \bmod q$  and  $\bar{b} = \langle \mathbf{w}, \mathbf{b} \rangle \bmod q$
- 4: Perform MITM attack on  $(\bar{\mathbf{A}}, \bar{\mathbf{b}})$  and get a set of candidates  $(\mathbf{s}_1^{(1)}, \mathbf{s}_2^{(1)})$  (i.e., perform Algorithm 2 until line 5)
- 5: Filter out the portion of the candidates that meets the condition  $\tilde{\mathbf{s}}_1 := \mathbf{s}_1^{(1)} + \mathbf{s}_2^{(1)} \in \mathcal{T}_1^r(w_r)$  (i.e., the first judgement condition of line 7 in Algorithm 2)
- 6: Hypothesis test on these candidates and output  $\mathbf{s}_1 := \mathbf{s}_1^{(1)} + \mathbf{s}_2^{(1)}$  with the highest score (i.e., replacing the second judgment condition of line 7 in Algorithm 2)

it is computed as  $T_{ours} = T_{BKZ}(n, r, \beta) + T_{MITM}(n, r, q, w, \sigma)$ , where  $T_{BKZ}$  is the time of performing BKZ algorithm with blocksize  $\beta$  on lattice  $\Lambda^\perp(\mathbf{A}_2)$  and  $T_{MITM}$  is the time of performing MITM2 (see Eq.(9)) to solve the new LWE instance defined in Eq.(11) with  $M$  selected according to Eq.(14). The success probability of the attack is  $p_{ours} = \sum_{h=0}^{\hat{w}/2} p_s(h) \cdot p_\pi(h)$ , where  $p_s(h), p_\pi(h)$  are given in Eq.(12)-(13).

## 6 Clarify the Distinguishing Issues

Recently, Ducas-Pulles [13] identified a contradiction in the heuristic analysis used in hybrid dual attacks for distinguishing BDD (Bounded Distance Decoding) instance from uniform ones. The issue mainly arises from the independence heuristic, which assumes that all the individual scores, given by dual vectors, are mutually independent. In addition, when searching for a BDD instance among many uniform ones, it becomes likely that some uniform instances may lie closer to the lattice than the correct BDD target when the number of uniform instances is large. These concerns have ignited a surge of interest in better understanding and analyzing (hybrid) dual attacks [41, 37, 38, 12].

In this section, we clarify that our hybrid dual attacks targeting FHE schemes are not affected by the above two issues. The main reason is that FHE schemes typically have small ratios  $\frac{\sigma}{q}$  and  $\frac{w}{n}$ , which allows for simple instances for hypothesis testing and a small number of hypothesis testing instances to be tested, respectively. Since the BDD target associated with correct guessing is easily distinguished, this in turn makes it possible to perform hypothesis testing with very few short vectors. This number is typically smaller than the lattice dimension, which means the vectors can be chosen to be linearly independent, thus satisfying the independence heuristic and avoiding the two aforementioned issues.

Next we clarify these issues from three perspectives. First we show that a uniform candidate associated with a wrong guessed partial secret  $\mathbf{s}_1$  is farther away from the lattice than the correct one with high probability. Then we show that for FHE-type parameters, the number of short vectors used for the hypothesis testing is smaller than the lattice dimension, ensuring their linear independence. At last, we experimentally verify that, for these parameters, the hypothesis testing succeeds with probability close to 1 when using multiple linearly independent short vectors.

### 6.1 Distinguishability

**Background.** The hybrid dual attack divides the LWE equation into  $\mathbf{A}_2 \mathbf{s}_2 + \mathbf{e} = \mathbf{b} - \mathbf{A}_1 \mathbf{s}_1 \bmod q$ , where  $\mathbf{A} = (\mathbf{A}_1, \mathbf{A}_2) \in \mathbb{Z}_q^{m \times r} \times \mathbb{Z}_q^{m \times (n-r)}$ , then guesses  $\mathbf{s}_1$  and uses the short vectors in dual lattice  $\Lambda^\perp(\mathbf{A}_2)$  to hypothesis test  $\mathbf{b} - \mathbf{A}_1 \mathbf{s}_1$ .

Now we will show that the uniform instances  $\mathbf{b} - \mathbf{A}_1 \mathbf{s}'_1$  associated with wrong guessing  $\mathbf{s}'_1$  are farther away from the lattice  $\Lambda(\mathbf{A}_2)$  than the BDD instance computed by the correct  $\mathbf{s}_1$  with high probability. Then regardless of the number of uniform instances is, it does not affect the fact that the correct  $\mathbf{s}_1$  gets the highest score in the hypothesis test.

There are two basic types of dual attacks that use different lattice constructions defined as follows and we will show this distinguishing issue in them respectively: the lattice in MR dual attack [32] with  $\Lambda_1^\perp(\mathbf{A}) = \{\mathbf{w} | \mathbf{w} \mathbf{A} = \mathbf{0} \bmod q\}$  and that in ADPS dual attack [4] with  $\Lambda_2^\perp(\mathbf{A}) = \{(\mathbf{w}, \mathbf{v}) | \mathbf{w} \mathbf{A} = \mathbf{v} \bmod q\}$ . The ADPS dual attack is now the prominent one in the bit-security estimation of practical LWE-base schemes since it can benefit from small secrets and the practical schemes are all of this type for the sake of efficiency.

**MR Dual Attack.** In this case, we use the short vectors in lattice  $\Lambda_1^\perp(\mathbf{A}_2) = \{\mathbf{w} | \mathbf{w} \mathbf{A}_2 = \mathbf{0} \bmod q\}$  to perform the hypothesis testing. We assume that the distribution of  $\mathbf{b} - \mathbf{A}_1 \mathbf{s}'_1 \bmod q$  is computationally indistinguishable from  $\mathcal{U}(\mathbb{Z}_q^m)$ , which is consistent with that of current literature of dual attacks, where  $\mathbf{s}'_1$  is a wrong guessing. Next, we demonstrate that no vector of the form  $\mathbf{b} - \mathbf{A}_1 \mathbf{s}'_1$  is closer to the lattice  $\Lambda(\mathbf{A}_2)$  than the vector associated with the correct guess  $\mathbf{s}_1$ . We refer to such a wrong guess  $\mathbf{s}'_1$  as a *W-solution* in the following.

We denote  $\alpha \in (0, 1)$  the radius of a BDD target within distance  $\|\mathbf{e}\|$  in lattice  $\Lambda(\mathbf{A})$ , i.e., we have

$$\|\mathbf{e}\| \lesssim \alpha \cdot \lambda_1(\Lambda(\mathbf{A})) \approx \alpha \cdot \sqrt{\frac{m}{2\pi e}} \cdot q^{1 - \frac{n}{m}}. \quad (16)$$

For a wrong guessing  $\mathbf{s}'_1$ , the corresponding  $\mathbf{e}'$  is uniform in  $\mathbb{R}^m / \Lambda(\mathbf{A}_2)$ , where  $\mathbf{e}'$  is defined by  $\mathbf{A}_2 \mathbf{s}'_2 + \mathbf{e}' = \mathbf{b} - \mathbf{A}_1 \mathbf{s}'_1 \bmod q$ . Since  $\mathbf{e}'$  is uniform, we get the following heuristic.

**Heuristic 1.** For a rational score function  $f_{score}$ ,  $\exists \epsilon \geq 1$ , s.t.  $\forall \mathbf{e}'$  which satisfies  $\|\mathbf{e}'\| \geq \epsilon \cdot \|\mathbf{e}\|$ , the probability of  $f_{score}(\mathbf{e}') \geq f_{score}(\mathbf{e})$  is negligible.

According to Heuristic 1,  $\mathbf{s}'_1$  might get higher score only when  $\|\mathbf{e}'\| < \epsilon \cdot \|\mathbf{e}\|$ , where  $\epsilon \geq 1$  is an approximate factor depending on the score function. Thus we divide the area of  $\|\mathbf{e}'\|$  into two cases: (1)  $\|\mathbf{e}'\| \leq \epsilon \cdot \alpha \cdot \lambda_1(\Lambda(\mathbf{A}))$  and (2)  $\|\mathbf{e}'\| > \epsilon \cdot \alpha \cdot \lambda_1(\Lambda(\mathbf{A}))$ . According to Heuristic 1,  $\mathbf{e}'$  corresponds to a W-solution must be in the first case.

Now even we assume  $\mathbf{s}_1 \leftarrow \mathcal{U}(\mathbb{Z}_q^r)$ , which maximizes the search space of  $\mathbf{s}_1$  to  $q^r$ , we still have that the expected number of W-solution is

$$\begin{aligned} \mathbb{E}(\#(\text{W-solution})) &\leq q^r \cdot \Pr[\|\mathbf{e}'\| \leq \epsilon \cdot \alpha \cdot \lambda_1(\Lambda(\mathbf{A}))] \\ &= q^r \cdot \left( \epsilon \cdot \alpha \cdot \frac{\lambda_1(\Lambda(\mathbf{A}))}{\lambda_1(\Lambda(\mathbf{A}_2))} \right)^m = (\epsilon\alpha)^m \end{aligned}$$

where the first “=” comes from Lemma 2 in which we take radius  $\epsilon \cdot \alpha \cdot \frac{\lambda_1(\Lambda(\mathbf{A}))}{\lambda_1(\Lambda(\mathbf{A}_2))}$ . Taking  $\|\mathbf{e}\| \approx \sqrt{m}\sigma$  in Eq.(16) leads to  $\alpha \approx \sqrt{2\pi e} \cdot \sigma \cdot q^{\frac{n}{m}-1}$ . Then we come to the contradictory regime  $\epsilon \cdot \sqrt{2\pi e} \cdot \sigma \cdot q^{\frac{n}{m}-1} \geq 1$ . Taking  $\epsilon$  heuristically, say  $\epsilon = 2$ , which is fairly large enough from experiments,  $2\sqrt{2\pi e} \cdot \sigma \cdot q^{\frac{n}{m}-1} < 1$  could always be satisfied for practical schemes. Then the expected number of W-solution is much less than 1. Therefore, the probability for a wrong guess  $\mathbf{s}'_1$  getting a higher score than the unique real  $\mathbf{s}_1$  is low.

*Remark 4.* We remark that the analysis in Section 5.1 of a recent work [37] is basically the  $\epsilon = 1$  case in our analysis. However, they mainly discussed the complementary area of the contradiction regime, but this regime in [13] is only a sufficient condition which causes that some uniform samples might get higher score than the BDD samples. We consider the impact of score functions to make the results more rigorous and generic.

**ADPS Dual Attack.** The above analysis suggests that the large number of wrong guessing would not disturb distinguishing the correct one when using MR dual lattice. However, most current dual attacks works over the ADPS dual lattice  $\Lambda_2^\perp(\mathbf{A}_2) = \{(\mathbf{w}, \mathbf{v}) | \mathbf{w}\mathbf{A}_2 = \mathbf{v} \bmod q\}$  because of the small secret, which leads to the equation  $\mathbf{w} \cdot (\mathbf{b} - \mathbf{A}_1\mathbf{s}_1) = \mathbf{w} \cdot (\mathbf{A}_2\mathbf{s}_2 + \mathbf{e}) = \langle \mathbf{v}, \mathbf{s}_2 \rangle + \langle \mathbf{w}, \mathbf{e} \rangle \bmod q$ , i.e., now the hypothesis testing cares not only  $\|\mathbf{e}\|$  but also  $\|\mathbf{s}_2\|$ . In this case, a W-solution means  $\|(\mathbf{s}'_2, \mathbf{e}')\| < \epsilon \cdot \|(\mathbf{s}_2, \mathbf{e})\|$ , where  $\mathbf{s}'_2$  and  $\mathbf{e}'$  are defined by  $\mathbf{b} - \mathbf{A}_1\mathbf{s}'_1 = \mathbf{A}_2\mathbf{s}'_2 + \mathbf{e}' \bmod q$  and  $\epsilon \geq 1$  is an approximate factor as defined in heuristic 1. This is to ensure that  $\langle \mathbf{v}, \mathbf{s}'_2 \rangle + \langle \mathbf{w}, \mathbf{e}' \rangle \lesssim \epsilon \cdot (\langle \mathbf{v}, \mathbf{s}_2 \rangle + \langle \mathbf{w}, \mathbf{e} \rangle)$ . Since the coordinates of  $(\mathbf{w}, \mathbf{v})$  are independent and identically distributed according to Section 2.2, it follows from  $\|(\mathbf{s}'_2, \mathbf{e}')\| < \epsilon \cdot \|(\mathbf{s}_2, \mathbf{e})\|$  that  $\|\mathbf{e}'\| < \epsilon \cdot \|\mathbf{e}\|$  and  $\|\mathbf{s}'_2\| \lesssim \epsilon \cdot \|\mathbf{s}_2\|$ .

For convenience, we denote  $\mathbf{s} = (\mathbf{s}_1, \mathbf{s}_2) \in [-t, t]^r \times [-t, t]^{n-r}$ , then the number of hypothesis testing candidates is  $T = (2t + 1)^r$ . The expected number of the candidates associated with wrong guessing fall into the ball of radius  $\epsilon \cdot \alpha \cdot \frac{\lambda_1(\Lambda(\mathbf{A}))}{\lambda_1(\Lambda(\mathbf{A}_2))}$  is  $\left( \epsilon \cdot \alpha \cdot \frac{\lambda_1(\Lambda(\mathbf{A}))}{\lambda_1(\Lambda(\mathbf{A}_2))} \right)^m \cdot (2t + 1)^r$ . Among these  $T$  candidates, the W-solution additionally requires  $\|\mathbf{s}'_2\| \lesssim \epsilon \cdot \|\mathbf{s}_2\|$  to ensure that  $\langle \mathbf{v}, \mathbf{s}'_2 \rangle + \langle \mathbf{w}, \mathbf{e}' \rangle \lesssim$

$\epsilon \cdot (\langle \mathbf{v}, \mathbf{s}_2 \rangle + \langle \mathbf{w}, \mathbf{e} \rangle)$ . Then it is clear that the expectation of the number of W-solution is

$$\begin{aligned} \mathbb{E}(\#(\text{W-solution})) &< (\epsilon \cdot \alpha \cdot \frac{\lambda_1(\Lambda(\mathbf{A}))}{\lambda_1(\Lambda(\mathbf{A}_2))})^m \cdot (2t+1)^r \cdot (\frac{\epsilon \cdot (2t+1)}{q})^{n-r} \\ &< (\frac{2t+1}{q})^n \cdot (\epsilon\alpha)^m \cdot \epsilon^{n-r} < (\epsilon \cdot \frac{2t+1}{q})^n \cdot (\epsilon\alpha)^m, \end{aligned}$$

as for wrong guessing  $\mathbf{s}'_1$ , the associated  $\mathbf{s}'_2$  is uniform in  $\mathbb{Z}_q^{n_2}$ . Then in this case  $\mathbb{E}(\#(\text{W-solution}))$  is even less than that in MR case due to  $\epsilon \cdot (2t+1) < q$ . That is to say, the probability of existing a wrong guessing  $\mathbf{s}'_1$  obtaining higher score than the correct  $\mathbf{s}_1$  becomes smaller. In conclusion, the probability of a wrong guessing getting higher score than the correct one is low, although the number of samples in hypothesis testing computed according to wrong guessing is large.

**Example.** We take CKKS as an example to show that for practical FHE schemes, no wrong guessing will gain higher score than the correct one. In other words, when performing our attack on CKKS, the parameters are not in the so-called contradictory regime. To this end, we first compute the number  $T$  of the uniform instances associated with wrong guessing and then compute the expected number of the uniform instances that fall in the ball with radius  $\mu = \frac{\sqrt{m} \cdot \sigma}{\text{GH}(\Lambda(\mathbf{A}_2))}$ , which is defined by the error  $\mathbf{e}$  of the BDD instance (see Lemma 2). The results are given in Table 2 that the expected number of uniform targets fall into the ball of radius  $\mu$  equal to  $T \cdot \frac{1}{\mu^m}$ , is far less than 1 for each case. Therefore, there is no uniform target closer to the lattice than the BDD target with overwhelm probability.

Table 2: The distinguishability of CKKS [11]

| Scheme        | $\log n$ | 13    |      |       | 14    |      | 15     |
|---------------|----------|-------|------|-------|-------|------|--------|
|               | $\log q$ | 120   | 130  | 155   | 185   | 250  | 620    |
| $\log T$      |          | 65    | 67   | 52    | 65    | 50   | 60     |
| $-\log \mu^m$ |          | 17643 | 6697 | 19822 | 24233 | 2274 | 109068 |

$\#(\text{uniform targets fall into the ball of radius } \mu) = T \cdot \frac{1}{\mu^m} \ll 1$  for each case.

## 6.2 Independence

A crucial heuristic used in the analysis of dual attacks is the assumption that all short vectors used in the hypothesis testing are independent. This does not hold when the attack needs an exponential number of short vectors. In this section, we show that, for FHE-type parameters, the number of short vectors needed for the hybrid dual attack does not exceed the lattice dimension, i.e.,  $M \leq \dim$ . This ensures all short vectors used in the hypothesis testing are indeed independent.

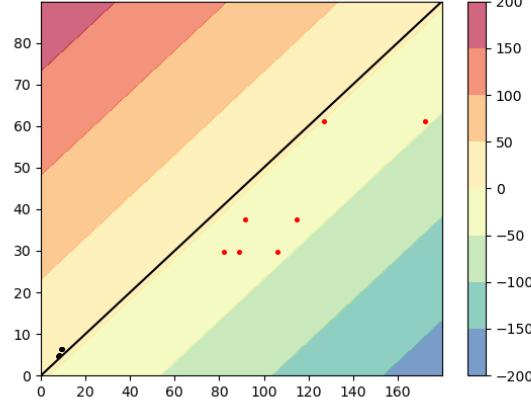


Fig. 3: Value of  $X$  for different  $\log(\frac{q}{\sigma})$  (x-axis) and  $\log(\delta_0^{2\dim} \cdot \sigma_s^{\frac{2(n-r)}{\dim}})$  (y-axis).

Recall that we choose  $M = \epsilon^{-2} \ln T = \exp(\frac{4\pi^2 \sigma^2 \ell^2}{q^2}) \cdot \ln T$  (see Section 5.2). First, let's establish an upper bound for  $\ln T$ . Since  $T$  denotes the number of candidates used in the hypothesis testing, it is clearly upper bounded by the overall runtime of the attack (as testing all  $T$  candidates is just one step in the attack). Given that the bit-security of practical schemes typically reaches up to 256 bits, we can conclude that  $T \leq 2^{256}$ . Furthermore, for practical FHE schemes,  $n \geq 2^{10}$ , which implies  $\ln T \leq 300 \ln 2 \leq 0.204n$ .

Next, we show that for FHE-type parameters, the exponential term  $X := \frac{4\pi^2 \sigma^2 \ell^2}{q^2}$  is bounded by 1. Therefore, we can achieve the desired bound with  $M \leq e \cdot \ln T \leq 0.55n \leq \dim$ . We compute  $X$  as  $X = \frac{4\pi^2 \sigma^2 \ell^2}{q^2} = \frac{4\pi^2 \sigma^2}{q^2} \cdot \delta_0^{2\dim} \cdot \left(\frac{q}{\sigma/\sigma_s}\right)^{\frac{2(n-r)}{\dim}} = 4\pi^2 \cdot \left(\frac{q}{\sigma}\right)^{\frac{n-r}{\dim}-2} \cdot \delta_0^{2\dim} \cdot \sigma_s^{\frac{2(n-r)}{\dim}}$ , where  $\sigma_s$  is the deviation of the secret and  $c = \frac{\sigma}{\sigma_s}$  is the scale factor. The dimension is given by  $\dim = m + (n-r) = \sqrt{\frac{(n-r) \log(q/c)}{\log(\delta_0)}}$  (the last “=” is from [32]). From a parameter-setting point of view, compared with LWE-based schemes in PQC, such as Kyber, FHE schemes generally have a much larger  $\frac{q}{\sigma}$  and  $n$  (resulting in a larger  $\dim$ ). Since  $\frac{n-r}{\dim} - 2 < 0$ , a larger  $\frac{q}{\sigma}$  leads to a smaller  $X$ . On the other hand, as  $\delta_0 > 1$ , a larger  $\dim$  leads to a larger  $X$ . Thus, the value of  $X$  is primarily determined by the interplay of these two opposing factors. Note that the factor  $\delta_0^{2\dim} \cdot \sigma_s^{2(n-r)/\dim}$  in  $X$  is also influenced by  $\delta_0$ , which is determined by  $\beta$ , the blocksize of BKZ (see Section 2.2). To eliminate the effect of  $\delta_0$  on  $X$  and ensure a fair comparison, we choose schemes with nearly the same bit-security (128 bits) to ensure similar  $\beta$  and  $\delta_0$ .

To demonstrate that  $X \leq 1$  holds for FHE-type parameters, we present the relation between  $X$  and two parameter sets,  $(q, \sigma)$  and  $(n-r, \sigma_s)$ , in Fig.3. In

this figure, the y-axis represents  $\log\left(\frac{q}{\sigma}\right)$  while the x-axis represents  $\log(\delta_0^{2\dim} \cdot \sigma_s^{\frac{2(n-r)}{\dim}})$ . Different color regions indicate varying ranges of  $\log(X)$ , with the black line corresponding to  $X = 1$ . To calculate the relation between the color regions and the x and y axes, we fix the security-level at 128 bits under RC.ADPS16 costmodel in the lattice-estimator [26]. Although the exact values of the x and y axes will vary depending on the security level, the trend is consistent for different security levels. In the figure, areas below the black line corresponds to  $X < 1$ , indicating that schemes in this region require a small number of short vectors. The farther a scheme is from the black line, the fewer short vectors it requires. On the contrary, areas above the black line indicate large numbers of short vectors are needed, with the black line serving as the dividing boundary.

In Fig.3, we label several practical schemes (both FHE and NIST-PQC LWE-based schemes) with approximately 128-bit security under RC.ADPS16 costmodel (in order to eliminate the effect of  $\delta_0$  in the y-axis term). The red points correspond to CKKS schemes [11] (with  $\log n = 13, 14$  cases, see Table 4), and HELib120-4096 and HELib80-4096 [19]. The black points represent NIST-PQC round-3 schemes [35]: Kyber512, LightSaber, Frodo640 and NTRUHRSS701Enc. From the figure, it is evident that PQC and FHE schemes are separated by the black line, with all the FHE schemes having  $X \leq 1$ . For LWE-based schemes in PQC, the hypothesis testing in the hybrid dual attacks needs exponential multiple short vectors, which causes the independence issue raised by Ducas-Pulles [13]. This separation also explains why current hybrid dual attacks are effective against for FHE schemes but not against LWE-based schemes in PQC. In Section 7, we present data of  $M$  for three typical cases using our new hybrid dual attack (see Table 5). In all instances,  $M$  is smaller than the dimension dim.

### 6.3 Hypothesis Testing Experiments

In this subsection, we experimentally verify that, for FHE schemes, it is sufficient to use  $N$  linear independent short vectors (computed using Eq.(14)) in the hypothesis testing to ensure a high success probability. In the experiments, we choose different parameters  $(n, q, w)$ , as shown in Table 3, and fix the error standard deviation at  $\sigma = 1.596$ . For convenience, we fix the number of given LWE instances by  $m = n$ , i.e.,  $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_q^{n \times n} \times \mathbb{Z}_q^n$ . In addition, we fix the guessing dimension at  $r = 10$  and the candidate number of hypothesis testing  $T$  at its upper bound  $3^r = 59049$ . Then we construct the dual lattice over  $\mathbf{A}_2 \in \mathbb{Z}_q^{n \times (n-r)}$  as  $\Lambda_2^\perp(\mathbf{A}_2) = \{(\mathbf{w}, \mathbf{v}) | \mathbf{w}\mathbf{A}_2 = \mathbf{v} \bmod q\}$  and run BKZ with blocksize 3 on it to get the first  $M$  shortest linear independent vectors from the sieving oracle, where  $M$  is calculated from Eq.(14).

For each case, we calculate the success probability in theory  $p_{\text{pred}}$  using Eq.(15) and conduct 1000 experiments to determine the real success  $p_{\text{real}}$ . The results are shown in Table 3. We can see that  $p_{\text{real}}$  is very close to 1, indicating that the number of short vectors is sufficient for the hypothesis testing in nearly all cases. Furthermore,  $p_{\text{real}}$  consistently exceeds  $p_{\text{pred}}$ , as  $p_{\text{pred}}$  is calculated us-

Table 3: Hypothesis Testing Experiments

| Experiment Parameter |      |     |     | Probability       |                   |
|----------------------|------|-----|-----|-------------------|-------------------|
| $n$                  | $q$  | $w$ | $M$ | $p_{\text{pred}}$ | $p_{\text{real}}$ |
| 40                   | 509  | 10  | 28  | 91.75%            | 99.4%             |
| 45                   | 609  | 12  | 35  | 91.74%            | 99.2%             |
| 50                   | 809  | 12  | 38  | 91.77%            | 99.7%             |
| 55                   | 1109 | 14  | 39  | 91.82%            | 99.6%             |
| 60                   | 1409 | 14  | 45  | 91.81%            | 99.8%             |

ing the larger norm of the  $M$  short vectors, meaning it provides a lower bound for  $p_{\text{real}}$ .

## 7 Bit-security Estimation and Comparison

To assess the performance of our new attack on practical schemes, we compare ours with previous hybrid attacks on a series of parameters of CKKS [11] (which is a leading scheme widely used in many open-source implementations of homomorphic encryption) and a new accelerated BGV schemes in [29]. The bit-security estimation results are given in Table 4. For all schemes, we use their original costmodels of BKZ to ensure a fair comparison.

As is shown in Table 4, for all cases of CKKS [11], our attack works the best, and our attack greatly reduces the estimation results by 17 to 24 bits under BKZ-coreSVP-model. Although our attack does not invalidate any scheme (the claimed security is 80 bits), it is still noteworthy that for some schemes the results estimated by using our attack is already very close to 80 bits.

For schemes in [29], our new attack significantly reduces the bit-security for all schemes, placing each one clearly below the claimed security-level, with the largest discrepancy being 18 bits. Therefore, all schemes in [29] are invalid under these findings and will need re-parameterization.

In Appendix A, we present a comprehensive comparison of our new attack with the previous hybrid dual attacks (BLWZ [6], CHHS [10], and BLW [5]) using three typical cases from [5], where the improvements of our attack is 11-34 bits. We also provide a comparison with the latest hybrid primal attack from [20]. Our attack is on par with it, less than 3 bits away from each other.

## 8 Conclusion

In this work, we conducted a comprehensive analysis and comparison of four variants of Meet-LWE, utilizing different list constructions and hash functions.

Table 4: The bit-security estimation of practical FHE-schemes

| Results of CKKS [11] |          |     |     |     |     |     |     |
|----------------------|----------|-----|-----|-----|-----|-----|-----|
| Schemes              | $\log n$ | 13  |     |     | 14  |     | 15  |
|                      | $\log q$ | 120 | 130 | 155 | 185 | 250 | 620 |
| Bit-security         | Claimed  | 80  |     |     |     |     |     |
|                      | CHHS     | 117 | 111 | 99  | 137 | 116 | 102 |
|                      | BLW      | 127 | 123 | 116 | 163 | 158 | 117 |
|                      | OURS     | 97  | 91  | 82  | 113 | 97  | 83  |

The LWE instances used in these schemes have Gaussian error with standard deviation 3.2 and sparse ternary secret with Hamming weight 64.

| Results of the schemes in [29] |          |       |       |       |       |       |       |       |       |       |       |
|--------------------------------|----------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| Schemes                        | $n$      | 36960 | 55080 | 54096 | 44100 | 50112 | 36960 | 55080 | 54096 | 44100 | 50112 |
|                                | $\log q$ | 63.8  | 69.4  | 64.2  | 66.3  | 74.6  | 64.5  | 60.7  | 64.7  | 66.8  | 75.1  |
|                                | $w$      | 24    | 22    | 22    | 22    | 22    | 12    | 12    | 12    | 12    | 12    |
|                                | $\sigma$ | 626.3 | 761.6 | 753.4 | 680.3 | 720.8 | 626.3 | 761.6 | 753.4 | 680.3 | 720.8 |
| Bit security                   | Level    | 128   |       |       |       |       | 80    |       |       |       |       |
|                                | Claimed  | 136   | 134.1 | 135.5 | 130.2 | 130.6 | 81.3  | 87.3  | 87.4  | 83.3  | 83    |
|                                | OURS     | 122   | 117   | 116   | 112   | 110   | 65    | 72    | 73    | 67    | 68    |

Based on this analysis, we proposed a new hybrid dual attack using a generalization of Meet-LWE and an efficient decoder in the hypothesis testing. We clarified that for FHE-type parameters our attack requires only a small number of short vectors generated by the BKZ algorithm and hence does not depend on the independence heuristics and is out of the so-called contradictory regime defined by Ducas-Pulles [13]. Lastly, we compared our new attack against previous hybrid dual attacks across various LWE instances with FHE-type parameters. The results show that our attack significantly outperforms earlier hybrid dual attacks and remains competitive with the latest hybrid primal attack.

## Appendix

### A Further Comparison with Previous Hybrid Attacks

#### A.1 Three Typical Cases in [5]

We start with the three typical cases given by Bi-Lu-Luo-Wang [5], which was used to illustrate the advantages and disadvantages of their attack BLLW compared with other attacks. The estimation results are given in Table 5, where the results of (hybrid) dual attacks [4–6, 10] in the first four columns are from [5]. Following [5] we assume that BKZ with blocksize  $\beta$  and basis dimension  $d$  takes  $T_{\text{BKZ}}(d, \beta) = 8d \cdot 2^{0.292\beta + 16.4}$  and we use the amortizing cost model for the output vectors of BKZ in [2].

To get a clearer comparison, for each case we present three different results using different variants of our attack in the column “Ours”. The first one “Test1” uses the trivial decoder and MITM2, and we enforce the algorithm to use the same  $r$  and  $\beta$  as BLLW to get a direct comparison with it. The second one “Opt1” also uses the trivial decoder and MITM2, but it chooses the optimal attack parameters to find the best estimation results. The last one “Opt2” is our new attack, which uses the simple decoder and MITM2, and chooses the optimal attack parameters.

For each case, in addition to the estimation results, we also present the cost of both BKZ and guessing, the success probability and the parameters: number of LWE instances  $m$  (which is computed according to Section 5.1), blocksize  $\beta$ , guessing dimension  $r$ , the mapping dimension of level-1 lists  $k$  (if any), and the number  $M$  of short vectors needed for the attack. Note that in all three cases and across all variants of our attack,  $M$  is consistently upper bounded by 400, and thus remains smaller than  $\dim$ . Specifically, for our new attack “Opt2”,  $M$  is always less than 150.

**Case 1.** For case 1, BLLW was the best for this case before introducing our new attack, and our new attack (the last column) improves the result of BLLW by 17 bits. The column “Test1” shows that MITM2 consumes less time (167 vs 170) and gains better success probability (22 vs 30) compared with BLLW. This means that MITM2 has greater capability than MITM1, which is consistent with the analysis in Section 3. Compared with “Test1”, “Opt1” balances the time of BKZ and guessing by using a slightly larger guessing dimension  $r$  (2251 vs 2245) and a smaller blocksize  $\beta$  (470 vs 483) such that  $T_{\text{BKZ}} \approx T_{\text{Guess}}$ . In addition, it also optimizes the balance between the time and success probability, i.e., the time of BKZ is decreased while the success probability is also decreased. The difference between “Opt1” and “Opt2” is the hypothesis testing algorithm which leads to a further increase in  $r$  (2260 vs 2251) and a decrease in  $\beta$  (430 vs 483). It is noteworthy that the main improvement of our new attack comes from the different between “Opt1” and “Opt2” due to the hypothesis testing algorithm.

Table 5: Three typical cases in [5]

| Case 1. $(\log n, \log q, w, \sigma) = (12, 50, 128, 3.2)$ |         |      |       |      |            |       |      |            |
|------------------------------------------------------------|---------|------|-------|------|------------|-------|------|------------|
| Attack                                                     |         | Dual | BLLWZ | CHHS | BLLW       | Ours  |      |            |
|                                                            |         | [4]  | [6]   | [10] | [5]        | Test1 | Opt1 | Opt2       |
| Cost<br>(bit)                                              | Attack  | 299  | 215   | 221  | <b>201</b> | 191   | 189  | <b>185</b> |
|                                                            | BKZ     | 299  | 166   | 183  | 175        | 175   | 169  | 157        |
|                                                            | Guess   | -    | 166   | 181  | 174=151+23 | 176   | 169  | 157        |
|                                                            | Prob.   | -    | 49    | 38   | 26         | 15    | 20   | 28         |
| Parameter                                                  | $m$     | 3572 | 2289  | 2343 | 2022       | 2261  | 2195 | 2070       |
|                                                            | $r$     | -    | 1689  | 2245 | 2245       | 2245  | 2251 | 2260       |
|                                                            | $\beta$ | 912  | 494   | 520  | 492        | 492   | 470  | 430        |
|                                                            | $k$     | -    | -     | -    | 1          | 43    | 39   | 16         |
|                                                            | $M$     | -    |       |      |            | 348   | 326  | 112        |

| Case 2. $(\log n, \log q, w, \sigma) = (13, 200, 128, 3.2)$ |         |      |       |            |            |       |      |            |
|-------------------------------------------------------------|---------|------|-------|------------|------------|-------|------|------------|
| Attack                                                      |         | Dual | BLLWZ | CHHS       | BLLW       | Ours  |      |            |
|                                                             |         | [4]  | [6]   | [10]       | [5]        | Test1 | Opt1 | Opt2       |
| Cost<br>(bit)                                               | Attack  | 140  | 124   | <b>120</b> | 204        | 198   | 111  | <b>109</b> |
|                                                             | BKZ     | 140  | 112   | 112        | 198        | 197   | 104  | 100        |
|                                                             | Guess   | -    | 110   | 112        | 200=121+79 | 187   | 105  | 99         |
|                                                             | Prob.   | -    | 12    | 9          | 4          | 1     | 6    | 9          |
| Parameter                                                   | $m$     | 7995 | 6834  | 6499       | 10633      | 10284 | 6101 | 5880       |
|                                                             | $r$     | -    | 1650  | 2050       | 1637       | 1637  | 2456 | 2500       |
|                                                             | $\beta$ | 365  | 270   | 269        | 563        | 563   | 244  | 230        |
|                                                             | $k$     | -    | -     | -          | 1          | 13    | 47   | 13         |
|                                                             | $M$     | -    |       |            |            | 374   | 215  | 57         |

| Case 3. $(\log n, \log q, w, \sigma) = (10, 20, 192, 3.2)$ |         |      |            |      |            |       |      |      |
|------------------------------------------------------------|---------|------|------------|------|------------|-------|------|------|
| Attack                                                     |         | Dual | BLLWZ      | CHHS | BLLW       | Ours  |      |      |
|                                                            |         | [4]  | [6]        | [10] | [5]        | Test1 | Opt1 | Opt2 |
| Cost<br>(bit)                                              | Attack  | 188  | <b>175</b> | 299  | 228        | 226   | 198  | 185  |
|                                                            | BKZ     | 188  | 169        | 265  | 211        | 211   | 187  | 167  |
|                                                            | Guess   | -    | 169        | 265  | 211=146+65 | 180   | 187  | 164  |
|                                                            | Prob.   | -    | 5          | 34   | 18         | 15    | 11   | 18   |
| Parameter                                                  | $m$     | 918  | 847        | 1097 | 988        | 931   | 8666 | 788  |
|                                                            | $r$     | -    | 161        | 420  | 455        | 455   | 408  | 459  |
|                                                            | $\beta$ | 539  | 475        | 805  | 620        | 620   | 537  | 470  |
|                                                            | $k$     | -    | -          | -    | 7          | 8     | 44   | 44   |
|                                                            | $M$     | -    |            |      |            | 309   | 296  | 142  |

**Case 2.** For case 2, before introducing our attack, CHHS works the best and BLLW is much worse than the others. The main reason for the poor performance of BLLW is the time-consuming enumeration of the error (79 bits). This in turn is due to that the range  $E$  of the error  $\bar{e}$  of the new instance in Eq.(10) is usually

large when  $q$  is large and the value of  $E$  has a big influence on BLLW as it needs to enumerate some entries of the error  $\bar{e} \in [-E, E]$ . Our new attack solves exactly this error enumeration problem and it is not sensitive to the exact value of  $E$ . Thus, it works well in this case, even better than the previous optimal attack CHHS because of the high efficiency of MITM2, which can be seen from the comparison between “CHHS” and “Opt1”. In addition, since this case has very large  $q$ , both hypothesis tests used in “Opt1” and “Opt2” can have a good effect and hence “Opt2” has only 1 bit boost compared to “Opt1”.

**Case 3.** Case 3 has weight ratio  $\frac{w}{n} \approx 0.177$ , which is much larger than case 1 (0.031) and case 2 (0.015). This parameter setting falls in the less efficient area of the hash functions in MITM since the large weight ratio increases the deviation  $\rho$  of the error of the new LWE instance in the guessing-stage, where using MITM is even worse than exhaustive searching. As a result, BLLWZ, using exhaustive searching in the guessing-stage, works better than CHHS and BLLW, which use MITM. Actually, this set of parameter is closer to the LWE-based scheme in PQC than it is to FHE-type parameter from the point of view of the ratios  $\frac{w}{n}$  and  $\frac{\sigma}{q}$ , which is out of the scope of MITM-accelerated hybrid dual attacks (see Remark 5 and Section 6.2 for the discussion about this issue). Nevertheless, our new attack works much better than the previous two MITM-accelerated hybrid dual attacks (CHHS and BLLW); the improvements are 99 bits and 34 bits, respectively. In addition, our new attack is better than the standalone dual attack and is relatively close to the optimal one BLLWZ.

*Remark 5.* A necessary condition for the hash function to distinguish  $E$ -near pairs from uniform ones is that the collision probability  $p_c(E, B)$  must be larger than  $\frac{1}{\theta}$ , the collision probability for random pairs. This fundamental limitation of the hash function is in our view the key reason for the ineffectiveness of MITM-accelerated hybrid attacks on LWE-based PQC schemes. For instance, consider the case where  $B = \frac{q}{2}$  and hence  $\frac{1}{\theta} = \frac{1}{2}$ . Let  $\rho$  denote the error deviation of the LWE instance being solved via the MITM step in hybrid dual attacks (see Eq.(11) for more details). Then the condition  $p_c(\rho, B) > \frac{1}{2}$  derives an upper bound on  $\rho$ . For FHE parameters, since  $\frac{\sigma}{q}$  is very small, its  $\rho$  satisfies the required bound, making such attacks viable. However, for standard LWE-based schemes in PQC, the ratio  $\frac{\sigma}{q}$  is typically larger, and the corresponding value of  $\rho$  exceeds this threshold. This explains why MITM-accelerated hybrid attacks have not proven effective in the PQC setting.

## A.2 Comparison with the Hybrid Primal Attack [20]

We compare our new attack with hybrid primal attack HKLS [20], which combines primal attack with MITM2. As the source code of [20] is not publicly available, in Table 6 we compare the result of our attack (the last row) with the results given by [20] under the refined BKZ model [26]. From the table we can see an interesting match between different variants of primal attacks and

dual attacks. The difference between the results of primal attack and dual attack is within 2 bits, the difference between the results of primal+MITM1 and dual+MITM1 is within 4 bits, and the difference between the results of primal+MITM2 (HKLS) and dual+MITM2 (our attack) is within 3 bits.

Table 6: The bit-security estimation of the schemes in [20]

| Schemes | $\log n$      | 15         |            | 16         |            |
|---------|---------------|------------|------------|------------|------------|
|         | $(\log q, w)$ | (699,128)  | (768,192)  | (1553,192) | (1450,64)  |
| Primal  | Primal        | 161        | 145        | 145        | 155        |
|         | Primal+MITM1  | 133        | 130        | 130        | 111        |
|         | Primal+MITM2  | 127        | <b>126</b> | <b>127</b> | <b>105</b> |
| Dual    | Dual          | 162        | 146        | 146        | 157        |
|         | Dual+MITM1    | 133        | 133        | 136        | 108        |
|         | Dual+MITM2    | <b>124</b> | 129        | <b>127</b> | 106        |

## References

1. Albrecht, M., Chase, M., Chen, H., Ding, J., Goldwasser, S., Gorbunov, S., Halevi, S., Hoffstein, J., Laine, K., Lauter, K., Lokam, S., Micciancio, D., Moody, D., Morrison, T., Sahai, A., Vaikuntanathan, V.: Homomorphic encryption security standard. Tech. rep., HomomorphicEncryption.org (2018)
2. Albrecht, M.R.: On dual lattice attacks against small-secret LWE and parameter choices in helib and SEAL. In: EUROCRYPT. vol. 10211, pp. 103–129 (2017)
3. Albrecht, M.R., Shen, Y.: Quantum augmented dual attack. Cryptology ePrint Archive, Paper 2022/656 (2022)
4. Alkim, E., Ducas, L., Pöppelmann, T., Schwabe, P.: Post-quantum key exchange - A new hope. In: 25th USENIX. pp. 327–343 (2016)
5. Bi, L., Lu, X., Luo, J., Wang, K.: Hybrid dual and meet-LWE attack. In: ACISP. vol. 13494, pp. 168–188 (2022)
6. Bi, L., Lu, X., Luo, J., Wang, K., Zhang, Z.: Hybrid dual attack on LWE with arbitrary secrets. Cybersecurity **5**(1), 15 (2022)
7. Bossuat, J.P., Troncoso-Pastoriza, J., Hubaux, J.P.: Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In: Applied Cryptography and Network Security. pp. 521–541 (2022)
8. Carrier, K., Shen, Y., Tillich, J.: Faster dual lattice attacks by using coding theory. Cryptology ePrint Archive, Paper 2023/1750 (2022)
9. Chen, Y., Nguyen, P.Q.: BKZ 2.0: Better lattice security estimates. In: ASIACRYPT. vol. 7073, pp. 1–20 (2011)
10. Cheon, J.H., Hhan, M., Hong, S., Son, Y.: A hybrid of dual and meet-in-the-middle attack on sparse and ternary secret LWE. IEEE Access **7**, 89497–89506 (2019)
11. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: ASIACRYPT. vol. 10624, pp. 409–437 (2017)

12. Ducas, L., Pulles, L.N.: Accurate score prediction for dual-sieve attacks. *Cryptology ePrint Archive*, Paper 2023/1850 (2023)
13. Ducas, L., Pulles, L.N.: Does the dual-sieve attack on learning with errors even work? In: *CRYPTO*. vol. 14083, pp. 37–69 (2023)
14. Espitau, T., Joux, A., Kharchenko, N.: On a dual/hybrid approach to small secret LWE - A dual/enumeration technique for learning with errors and application to security estimates of FHE schemes. In: *INDOCRYPT*. vol. 12578, pp. 440–462 (2020)
15. Esser, A., Girme, R., Mukherjee, A., Sarkar, S.: Memory-efficient attacks on small LWE keys. In: *ASIACRYPT*. vol. 14441, pp. 72–105 (2023)
16. Glaser, T., May, A.: How to enumerate LWE keys as narrow as in kyber/dilithium. In: *CANS*. vol. 14342, pp. 75–100 (2023)
17. Guo, Q., Johansson, T.: Faster dual lattice attacks for solving LWE with applications to *CRYSTALS*. In: *ASIACRYPT*. vol. 13093, pp. 33–62 (2021)
18. HEAAN: <https://github.com/snucrypto/HEAAN>
19. HELib: <https://github.com/homenc/HELib>
20. Hhan, M., Kim, J., Lee, C., Son, Y.: Let’s meet ternary keys on babai’s plane: A hybrid of lattice-reduction and meet-lwe. *Cryptology ePrint Archive*, Paper 2022/1473 (2022)
21. Hoffstein, J., Pipher, J., Silverman, J.H.: NTRU: A ring-based public key cryptosystem. In: *ANTS*. vol. 1423, pp. 267–288 (1998)
22. Homomorphic Encryption Standardization: <https://homomorphiccryptography.org>
23. van Hoof, I., Kirshanova, E., May, A.: Quantum key search for ternary LWE. In: *PQCrypto*. vol. 12841, pp. 117–132 (2021)
24. Kirshanova, E., May, A.: How to find ternary LWE keys using locality sensitive hashing. In: *IMACC*. vol. 13129, pp. 247–264 (2021)
25. Laarhoven, T., Walter, M.: Dual lattice attacks for closest vector problems (with preprocessing). In: *CT-RSA*. vol. 12704, pp. 478–502 (2021)
26. Lattice estimator.: <https://github.com/malb/lattice-estimator/>
27. Lattigo: <https://github.com/tuneinsight/lattigo>
28. Lee, E., Lee, J., Son, Y., Wang, Y.: Improved meet-LWE attack via ternary trees. *Cryptology ePrint Archive*, Paper 2024/824 (2024)
29. Ma, S., Huang, T., Wang, A., Wang, X.: Accelerating BGV bootstrapping for large  $p$  using null polynomials over  $\mathbb{Z}_{p^e}$ . In: *EUROCRYPT*. vol. 14652, pp. 403–432 (2024)
30. MATZOV: Report on the security of LWE: improved dual lattice attack (2022)
31. May, A.: How to meet ternary LWE keys. In: *CRYPTO*. vol. 12826, pp. 701–731 (2021)
32. Micciancio, D., Regev, O.: Lattice-based cryptography. In: *Post-Quantum Cryptography*. Springer, Berlin Heidelberg (2009). [https://doi.org/10.1007/978-3-540-88702-7\\_5](https://doi.org/10.1007/978-3-540-88702-7_5)
33. Nguyen, P.Q.: Boosting the hybrid attack on NTRU: torus LSH, permuted HNF and boxed sphere, <https://csrc.nist.gov/CSRC/media/Events/third-pqc-standardization-conference/documents/accepted-papers/nguyen-boosting-hybridboost-pqc2021.pdf>
34. Nick Howgrave-Graham, Joseph H. Silverman, W.W.: A meet-in-the-middle attack on an NTRU private key, [https://www.researchgate.net/publication/2906622\\_A\\_Meet-In-The-Middle\\_Attack\\_on\\_an\\_NTRU\\_Private\\_Key](https://www.researchgate.net/publication/2906622_A_Meet-In-The-Middle_Attack_on_an_NTRU_Private_Key)
35. NIST-PQC Round 3 submissions: <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-3-submissions>

- 36. PALISADE: <https://gitlab.com/palisade/palisade-release>
- 37. Pouly, A., Shen, Y.: Provable dual attacks on learning with errors. In: EUROCRYPT. vol. 14656, pp. 256–285 (2024)
- 38. Pouly, A., Shen, Y.: Discrete gaussian sampling for bkz-reduced basis. In: PQC. pp. 63–88 (2025)
- 39. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: STOC. pp. 84–93 (2009)
- 40. Son, Y., Cheon, J.H.: Revisiting the hybrid attack on sparse secret LWE and application to HE parameters. In: WAHC@CCS. pp. 11–20 (2019)
- 41. Wiemers, A., Ehlen, S.: A remark on the independence heuristic in the dual attack. Cryptology ePrint Archive, Paper 2023/1238 (2023)
- 42. Wunderer, T.: A detailed analysis of the hybrid lattice-reduction and meet-in-the-middle attack. J. Mathematical Cryptology **13**(1), 1–26 (2019)